

**SVEUČILIŠTE U SPLITU
FAKULTET ELEKTROTEHNIKE, STROJARSTVA I
BRODOGRADNJE**

DIPLOMSKI RAD

Prediktivni kripto valutni model

Ivan Filipović

Split, rujan 2024.



SVEUČILIŠTE U SPLITU
FAKULTET ELEKTROTEHNIKE, STROJARSTVA I
BRODOGRADNJE



Sveučilišni diplomski studij: **Računarstvo**
Smjer/Usmjerenje: /
Oznaka programa: 250
Akademska godina: 2023./2024.

Ime i prezime: **Ivan Filipović**
Broj indeksa: 0023138683

ZADATAK DIPLOMSKOG RADA

Naslov: PREDIKTIVNI KRIPTO VALUTNI MODEL

Zadatak: Proučiti različite metode predviđanja kripto valuta. Pronaći i preuzeti podatke o kripto valutama. Analizirati modele i organizacijske postavke kripto valuta te oblikovati i predobraditi prikupljene podatke kako bi se nad njima analizirali odabrani pristupi za predviđanje. Primijeniti model nad pripremljenim podacima i analizirati dobivene rezultate. Za realizaciju praktičnog dijela koristiti Python programski jezik te gotove biblioteke poput Pandas, scikit-learn i sl.

Rad predan:

Predsjednik
Odbora za diplomski rad:

Mentor:

prof. dr. sc. Linda Vicković

prof. dr. sc. Maja Štula

IZJAVA

Ovom izjavom potvrđujem da sam diplomski rad s naslovom Prediktivni kripto valutni model pod mentorstvom prof. dr. sc. Maje Štule pisao samostalno, primijenivši znanja i vještine stečene tijekom studiranja na Fakultetu elektrotehnike, strojarstva i brodogradnje, kao i metodologiju znanstveno-istraživačkog rada, te uz korištenje literature koja je navedena u radu. Spoznaje, stavove, zaključke, teorije i zakonitosti drugih autora koje sam izravno ili parafrazirajući naveo u diplomskom radu citirao sam i povezao s korištenim bibliografskim jedinicama.

Student

Ivan Filipović

Sadržaj

1	UVOD	1
2	STROJNO UČENJE	2
2.1	Ansambl metode	3
2.2	Neuronske mreže	4
2.2.1	Neuron.....	5
2.2.2	Aktivacijske funkcije	6
2.2.3	Duboke neuronske mreže	9
2.2.4	Vrste dubokih neuronskih mreža	11
3	KORIŠTENI ALGORITMI I EVALUACIJSKE METRIKE	12
3.1	Algoritmi	12
3.1.1	Linearna regresija	12
3.1.2	Slučajna šuma	14
3.1.3	Poboljšano gradijentno pojačanje	15
3.1.4	Dugoročno - kratkoročna memorija.....	19
3.2	Evaluacijske metrike	21
3.2.1	Srednja apsolutna pogreška	21
3.2.2	Srednja kvadratna pogreška.....	21
3.2.3	Korijen srednje kvadratne pogreške.....	22
3.2.4	Koeficijent determinacije	22
4	PRAKTIČNI DIO RADA	23
4.1	Korištene tehnologije	23
4.2	Analiza podataka.....	24
4.3	Predobrada podataka i <i>feature engineering</i>	26
4.4	Trening i rezultati linearne regresije	27

4.5	Trening i rezultati slučajne šume	30
4.6	Trening i rezultati poboljšanog gradijentnog pojačanja	33
4.7	Trening i rezultati dugoročno – kratkoročne memorije	36
5	USPOREDBA REZULTATA	41
6	ZAKLJUČAK	43
	Literatura	44
	Sažetak	45
	Summary	46
	Prilozi	47
	Kazalo slika	47

1 UVOD

Kriptovalute kao što i sam naziv sugerira su digitalne valute zaštićene kriptografijom. Bolje rečeno, koriste sustav enkripcije (eng. Blockchain) koji pruža sigurnost i transparentnost transakcija. Jedna od prvih nastalih kriptovaluta, bitcoin je i najpoznatija.

Bitcoin je stvoren s ciljem da se ponudi alternativa standardnom načinu plaćanja, ali danas je dobio mnogo širu dimenziju primjene. Jedna od njih je kripto ulaganje. Kao glavna prednost bitcoina ističe se decentraliziranost, odnosno nepostojanje centralnog tijela koje njime upravlja. Negativna strana je njegova volatilnost (nagla promjena cijene) što predstavlja problem brojnim kripto ulagačima. Kripto ulagači koriste različite tehnike kako bi pokušali predvidjeti buduću cijenu bitcoina. Takvu mogućnost pronašli su u primjeni strojnog učenja.

Strojno učenje je postupak u kojem računalno iz mnoštva podataka „izvlači“ uzorke i veze među podacima te na temelju njih donosi zaključke o mogućim ishodima. Kroz ovaj diplomski rad obrađena su četiri algoritma strojnog učenja nad istim setom podataka o bitcoinu.

U drugom poglavlju predstavljen je kratki uvod u strojno učenje. Objašnjeni su neki od osnovnih pojmova te hijerarhija strojnog učenja. Opsežnije su obrađene podskupine strojnog učenja kojima pripadaju algoritmi korišteni kroz ovaj diplomski rad.

Treće poglavlje daje kratki uvid za svaki pojedinačni korišteni algoritam te evaluacijske metrike prema kojima su evaluirani rezultati predviđanja.

Četvrto poglavlje opisuje proces izrade praktičnog dijela rada. Na kraju su uspoređeni rezultati algoritama s ciljem pronalaska najprikladnijeg za već navedenu svrhu.

U zaključku su sumirani uvidi stečeni kroz ovaj rad.

2 STROJNO UČENJE

Strojno učenje (eng. Machine Learning) je grana umjetne inteligencije koja se temelji na statistici, odnosno prikupljanju mnoštva podataka iz kojih se izvlače relevantne informacije. Da ne bismo morali ručno prebirati na tisuće podataka i tražiti analogiju među njima algoritmi strojnog učenja će to učiniti umjesto nas. Dakle, primarna uloga algoritma strojnog učenja je naučiti odnose među podacima i na temelju njih predviđati.

Spektar primjene strojnog učenja poprima sve veći opseg. Neke od domena uporabe su:

- Zdravstvo – detekcija tumora i analiza DNA
- Financije – detekcija sumnjivih transakcija/prevara
- Marketing – preporuka proizvoda na temelju korisničkih preferenci
- Automobilska industrija – omogućavanje autonomne vožnje
- Energetika – optimizacija potrošnje

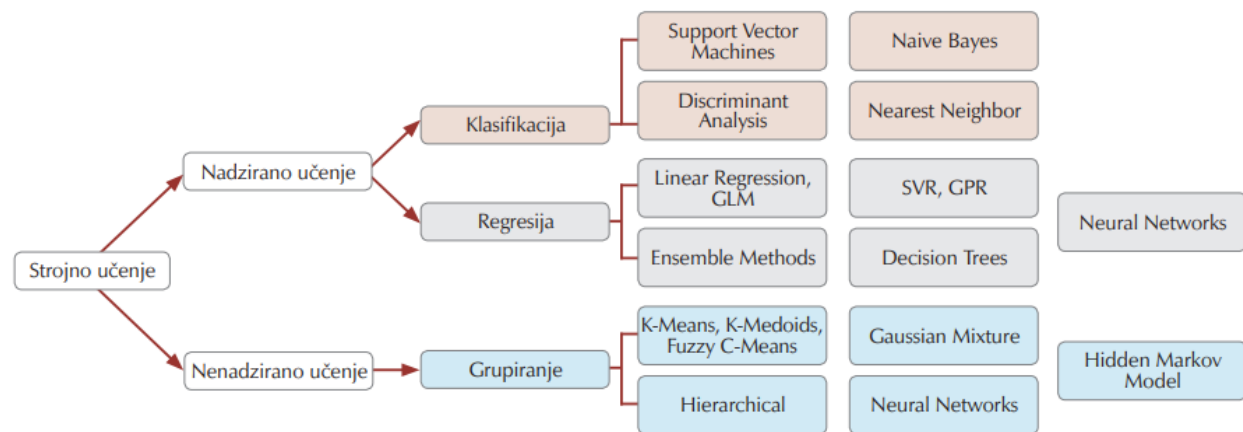
Osnovna podjela strojnog učenja je na **nenadzirano** i **nadzirano učenje**.

Nenadzirano učenje je vrsta strojnog učenja koja bez ljudske intervencije pronalazi skrivene obrasce unutar neoznačenih ulaznih podataka. Takvo učenje stvara svoja pravila pomoću kojih grupira podatke sa sličnim osobinama.

Nadzirano učenje je tip strojnog učenja gdje se model uči na temelju poznatih parova podataka ulaz-izlaz. Nadzirano učenje se može dodatno podijeliti na **klasifikaciju** i **regresiju**. **Klasifikacijom** se predviđa pripadnost nekoj od diskretnih klasa. Jedan od primjera uporabe klasifikacije je aplikacija „Google Lens“ koja prepoznaje različite objekte s fotografije. S druge strane **regresija** se koristi za predviđanje kontinuirane varijable kao što je promjena temperature, potrošnja goriva automobila, cijena dionica itd. Stoga, da se zaključiti da je upravo **regresija** područje na kojemu će se temeljiti ostatak ovoga rada. [1][2]

Kroz rad su obrađena četiri modela za rješavanje regresijskog problema. To su, linearna regresija (eng. Linear Regression) i dva modela koji pripadaju skupini ansambl modela, tj. slučajna šuma (eng. Random Forest) i ekstremno poboljšano gradijentno pojačanje (eng. Extreme Gradient Boosting, XGBoost). Posljednji model odnosno dugoročno-kratkoročna memorija (eng. Long

Short - Term Memory, LSTM) pripada skupini modela rekurentne neuronske mreže (eng. Recurrent Neural Network, RNN). Kroz nastavak rada će se ukratko pojasniti prethodni pojmovi, a za lakše razumijevanje hijerarhije modela može poslužiti slika 2-1.



Slika 2-1 Hijerarhija strojnog učenja [1]

2.1 Ansambl metode

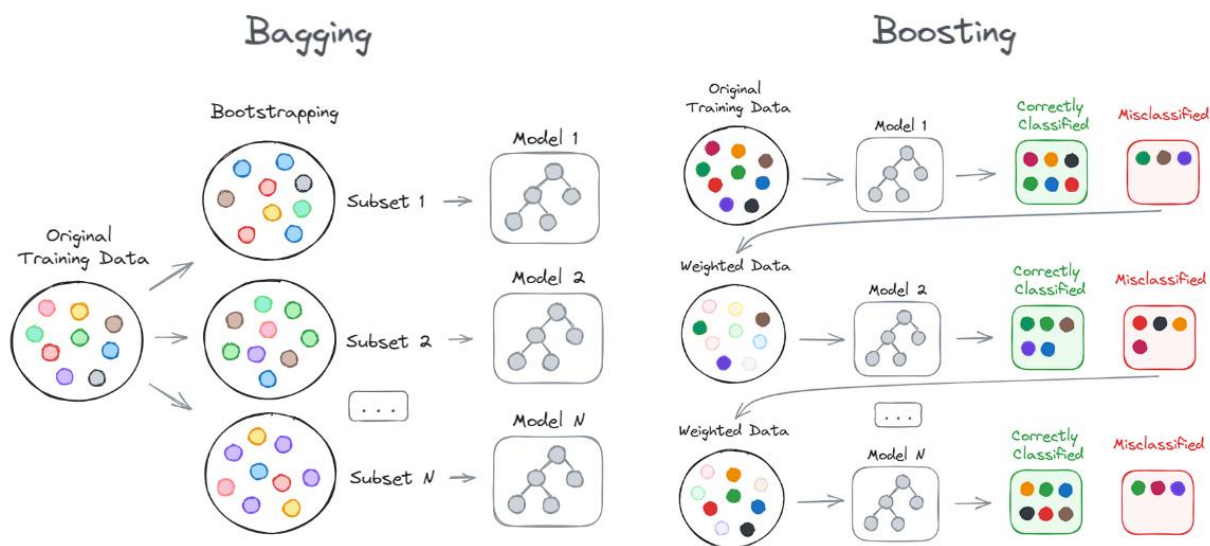
Ansambl metode (eng. Ensemble methods) su tehnike strojnog učenja koje se temelje na kombiniranju predikcija više modela. Međusobnim nadopunjavanjem modela smanjuje se šansa za prenaučenosť (eng. Overfitting) i postiže se bolja generalizacija podataka. Neke od najpopularnijih ansambl metoda su **pakiranje** (eng. Bootstrap Aggregating, Bagging) i **pojačavanje** (eng. Boosting).

Pakiranje je metoda koja se bazira na učenju više modela na različitim podskupovima podataka odabranih slučajnim uzorkovanjem s ponavljanjem. Konačna predikcija će se dobiti računanjem prosjeka pojedinačnih predikcija svih modela u slučaju regresije, tj. većinskim glasanjem u slučaju klasifikacije. Primjer ovakve metode je slučajna šuma koja kombinira više regresijskih stabala.

Druga metoda je **pojačavanje** koja uključuje sekvencijalno treniranje modela pri čemu svaki idući model uči na greškama prethodnog modela i pokušava ih ispraviti. Konačna predikcija se dobiva sumiranjem predikcija svih modela s pripadnim težinama. Primjer ovakve metode je ekstremno

poboljšano gradijentno pojačanje koje također kombinira više regresijskih stabala, ali na drugi način za razliku od slučajne šume. [3]

Slika 2-2 prikazuje grafičku usporedbu ovih dviju metoda.



Slika 2-2 Ansambl metode [4]

„Bagging“ metoda će nasumično cijeli set podataka podijeliti na manje podskupove i proslijediti ih različitim modelima. Klasifikacija će se konačno odrediti većinskim glasanjem. S druge strane „Boosting“ metoda će odmah cijeli set podataka proslijediti prvom modelu koji će svoje rezultate klasifikacije proslijediti drugom modelu. Drugi model će pokušati ispraviti greške prethodnog i tako redom do kraja.

2.2 Neuronske mreže

Neuronske mreže su skupina algoritama inspirirana građom i funkcijom ljudskoga mozga. Prvi takav rad o umjetnim neuronskim mrežama publicirali su McChulloh i Pitts još davne 1943. godine. Predstavili su jednostavan model neurona koji poput biološkog neurona obrađuje signale putem sinaptičke i somatske operacije.

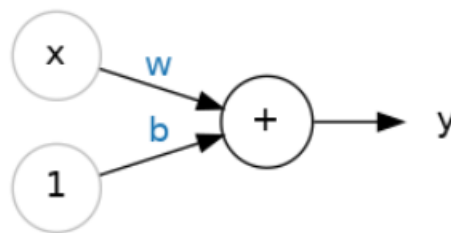
Neuronska mreža sastoji se od tri vrste međusobno povezanih slojeva: ulaznog, skrivenog i izlaznog sloja. Ulazni sloj prima podatke ulaznih veličina koje prosljeđuje do prvog skrivenog

sloja. Skriveni sloj zatim obrađuje podatke te ih prosljeđuje do izlaznog sloja koji daje konačan rezultat.

Razlog zašto se skriveni sloj zove „skriveni“ je jer se njegov izlaz zapravo nikada ne vidi. Svaki skriveni sloj se sastoji od čvorova ili neurona koji izvode jednostavnu matematičku operaciju na ulazu i prosljeđuje rezultat idućem sloju. [3][5]

2.2.1 Neuron

Najjednostavnija mreža se sastoji od jednog neurona s jednim ulazom. Takav neuron se može vidjeti na slici 2-3.



Slika 2-3 Neuron s jednim ulazom [6]

Ovaj neuron ima ulaz „x“ koji je s njim povezan putem veze s težinom „w“. Dakle, za bilo koji ulaz „x“ ono što dolazi do neurona je zapravo umnožak

$$w * x \quad (1)$$

Osim težine „w“ postoji i posebna težina koja se zove pristranost (eng. Bias), označena slovom „b“. Uz pristranost ne dolazi ulazna veličina jer pristranost služi neuronu da modificira izlaz neovisno o ulazima. Stoga, vrijedi

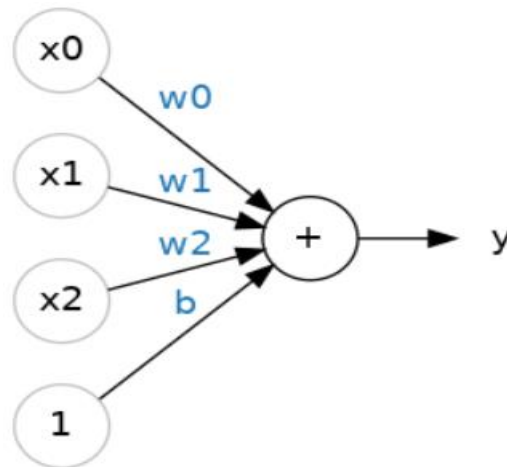
$$1 * b = b \quad (2)$$

Razlog zašto su težine bitne je jer njihovim namještanjem mreža uči o podacima i tako pronalazi funkciju koja će za zadane ulaze najbolje predviđati mogući izlaz. Kao izlaznu vrijednost ovaj neuron ima „y“. Izlaz se dobiva tako što neuron sumira sve vrijednosti koje prima putem svojih veza. To se može predočiti izrazom:

$$y = w * x + b \quad (3)$$

Da se uočiti da izraz (3) predstavlja jednadžbu pravca gdje je „w“ nagib pravca, a „b“ odsječak na osi y. To znači da ovakva neuronska mreža za ulaz „x“ može predviđati isključivo linearno povezane izlaze „y“.

Jedan neuron može primati više ulaza. Takav neuron prikazuje slika 2-4.



Slika 2-4 Neuron s više ulaza [6]

Ovaj neuron ima tri ulaza „x0“, „x1“ i „x2“ uz težine „w0“, „w1“ i „w2“ te pristranost „b“ što se može predstaviti izrazom

$$y = w0 * x0 + w1 * x1 + w2 * x2 + b \quad (4)$$

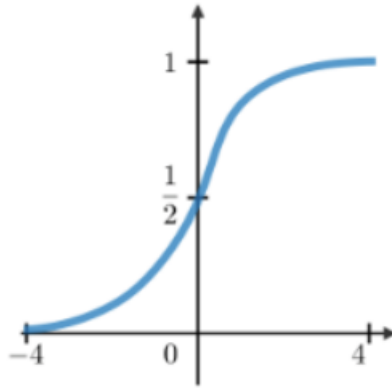
Izraz (4) predstavlja jednadžbu hiperravnine što znači da ovakva neuronska mreža i dalje može predviđati samo linearne veze između ulaza i izlaza. [6]

2.2.2 Aktivacijske funkcije

Neuronska mreža može imati i više skrivenih slojeva te se takva mreža zove **duboka neuronska mreža**. No, bez obzira na povećani broj skrivenih slojeva takva mreža ne može otkriti nelinearne korelacije među podacima. Dakle, potrebno je dodatno izmijeniti izlazne podatke skrivenih slojeva. To se postiže **aktivacijskim funkcijama**. Aktivacijske funkcije se primjenjuju na izlaz svakog neurona nakon što izračuna težinsku sumu ulaza. Neke od aktivacijskih funkcija su:

- Sigmoidalna funkcija – raspon izlaznih vrijednosti je od 0 do 1

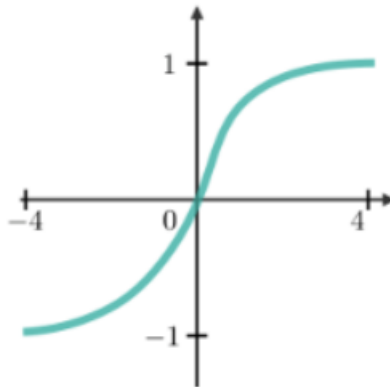
$$f(y) = \frac{1}{1 + e^{-y}} \quad (5)$$



Slika 2-5 Sigmoidalna funkcija [7]

- Tanh funkcija – raspon izlaznih vrijednosti je od -1 do 1

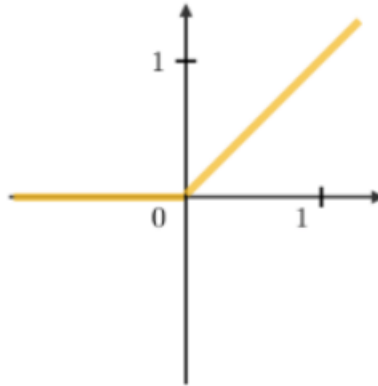
$$f(y) = \frac{2}{1 + e^{-2y}} - 1 \quad (6)$$



Slika 2-6 Tanh funkcija [7]

- ReLU funkcija – raspon izlaznih vrijednosti je od 0 do pozitivno beskonačno

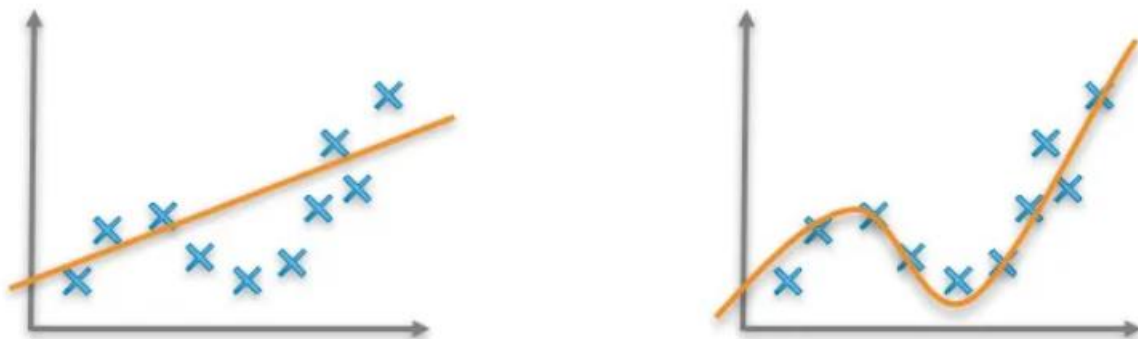
$$f(y) = \max(0, y) \quad (7)$$



Slika 2-7 ReLU funkcija [7]

Primjena aktivacijske funkcije na neurone koji kao izlaz daju pravce će učiniti da se taj pravac malo po malo „savija“ te poprima različite oblike funkcija koje mogu predviđati nelinearne veze. [2][7]

Slika 2-8 prikazuje jednostavan primjer kako bi to otprilike izgledalo u praksi. Na grafu lijevo se može vidjeti pravac koji ne može „pratiti“ stvarne vrijednosti dok krivulja desno poprilično precizno simulira te iste vrijednosti.



Slika 2-8 Neuronska mreža bez (lijevo) i s aktivacijskom funkcijom (desno) [8]

2.2.3 Duboke neuronske mreže

Duboke neuronske mreže (eng. Deep Neural Networks, DNN) su neuronske mreže koje između ulaznog i izlaznog sloja mogu imati od tri pa do više stotina skrivenih slojeva. Najveća prednost dubokih neuronskih mreža leži u njihovoj sposobnosti da prepoznaju hijerarhiju značajki. Mana im je da zahtijevaju velike količine podataka i računalnih resursa.

U nastavku će se matematičkim putem ukratko objasniti način rada dubokih neuronskih mreža.

Neka su slojevi neuronske mreže označeni indeksima $l \in \{1, \dots, L\}$ gdje je L ukupan broj slojeva bez ulaznog sloja te je svaki sloj opisan matricom težina $\mathbf{W}^{(l)}$ i vektorom pomaka $\mathbf{b}^{(l)}$. Na primjer $\mathbf{W}^{(1)}$ i $\mathbf{b}^{(1)}$ pripadaju sloju $l = 1$, $\mathbf{W}^{(2)}$ i $\mathbf{b}^{(2)}$ sloju $l = 2$, itd. Isto tako, neka su skriveni slojevi označeni s $\mathbf{q}^{(l)}$ te se svaki takav sloj sastoji od U_l neurona, $\mathbf{q}^{(l)} = [q_1^{(l)} \dots q_{U_l}^{(l)}]^T$ gdje dimenzije $U_1, \dots, U_{L-1}$ mogu biti različite za različite skrivene slojeve.

Tada, svaki skriveni sloj $\mathbf{q}^{(l-1)}$ se preslikava u idući skriveni sloj $\mathbf{q}^{(l)}$ prema formuli

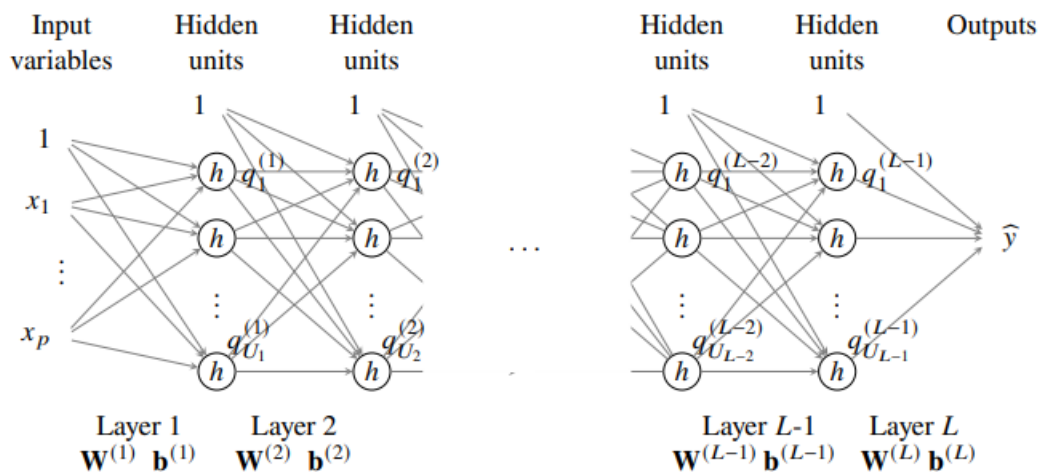
$$\mathbf{q}^{(l)} = h(\mathbf{W}^{(l)} \mathbf{q}^{(l-1)} + \mathbf{b}^{(l)}) \quad (8)$$

To znači da su slojevi složeni tako da je izlaz prvog skrivenog sloja $\mathbf{q}^{(1)}$ ulaz drugog skrivenog sloja $\mathbf{q}^{(2)}$, zatim njegov izlaz je ulaz trećeg skrivenog sloja, itd.

Slaganjem višestrukih skrivenih slojeva konstruira se duboka neuronska mreža. Duboka neuronska mreža s L slojeva može se matematički opisati:

$$\begin{aligned} \mathbf{q}^{(1)} &= h(\mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)}), \\ \mathbf{q}^{(2)} &= h(\mathbf{W}^{(2)} \mathbf{q}^{(1)} + \mathbf{b}^{(2)}), \\ &\vdots \\ \mathbf{q}^{(L-1)} &= h(\mathbf{W}^{(L-1)} \mathbf{q}^{(L-2)} + \mathbf{b}^{(L-1)}), \\ \hat{\mathbf{y}} &= \mathbf{W}^{(L)} \mathbf{q}^{(L-1)} + \mathbf{b}^{(L)} \end{aligned} \quad (9)$$

Slika 2-9 daje grafički prikaz prethodnih formula. Matrica težina $\mathbf{W}^{(1)}$ za prvi sloj $l = 1$ ima dimenzije $U_1 \times p$, a pripadni vektor pomaka $\mathbf{b}^{(1)}$ ima dimenziju U_1 . Kako je izlaz skalaran, matrica težina $\mathbf{W}^{(L)}$ ima dimenzije $1 \times U_{L-1}$, a vektor pomaka $\mathbf{b}^{(L)}$ ima dimenziju 1. Međuslojevi $l = 2, \dots, L - 1$, $\mathbf{W}^{(l)}$ imaju dimenziju $U_l \times U_{l-1}$ te odgovarajući vektor pomaka ima dimenziju U_l . Broj ulaznih jedinica p je početno zadan izabranim zadatkom. [2]



Slika 2-9 Duboka neuronska mreža [2]

2.2.4 Vrste dubokih neuronskih mreža

Duboke neuronske mreže se prema arhitekturi mogu podijeliti na **mreže s unaprijednim prijenosom podataka** (eng. Feed Forward Neural Networks, FFNN), **konvolucijske mreže** (eng. Convolutional Neural Networks, CNN) i **rekurentne neuronske mreže** (eng. Recurrent Neural Networks, RNN).

Neuronske mreže s unaprijednim prijenosom podataka najjednostavnija su vrsta dubokih neuronskih mreža. Naziv dolazi upravo iz toga jer se podatci prenose samo u jednom smjeru, od ulaznog sloja preko skrivenih slojeva pa do izlaznog sloja. Dakle, ne postoji povratna veza koja bi izlaze iz modela ponovno vraćala nazad u samoga sebe.

Konvolucijske neuronske mreže specijalizirane su za zadatke obrade slike. Kao bazu arhitekture im čine „FFNN“ mreže. Dodatno, koriste konvolucijske slojeve za otkrivanje lokalnih značajki te „pooling“ slojeve za smanjenje dimenzija podataka. Na kraju, potpuno povezani slojevi donose konačne odluke.

Rekurentne neuronske mreže pojednostavnjeno rečeno su „FFNN“ mreže s povratnim vezama. Kao što su konvolucijske mreže specijalizirane za obradu slike, rekurentne neuronske mreže su dizajnirane specifično za rad s nizovima podataka. Povratne veze im daju mogućnost pamćenja prethodnih stanja pa tako mogu raditi s podacima koji zavise o vremenu. Iako su RNN mreže vrlo moćne, mogu imati problema s učenjem dugoročnih ovisnosti. Taj problem je riješen u sofisticiranijim modelima kao što je već odabrani LSTM algoritam. [9]

3 KORIŠTENI ALGORITMI I EVALUACIJSKE METRIKE

3.1 Algoritmi

3.1.1 Linearna regresija

Kao što samo ime sugerira linearna regresija se koristi za rješavanje regresijskih problema. Ovakav model uzima vektor $\mathbf{x} \in R^n$ kao ulaznu vrijednost te kao izlaz daje skalarnu vrijednost $y \in R$. Neka je $\hat{y}$ vrijednost koju model predviđa, tada vrijedi

$$\hat{y} = \mathbf{w}^T \mathbf{x} \quad (10)$$

gdje je $\mathbf{w} \in R^n$ vektor parametara. Parametri su bitne vrijednosti kojima se kontrolira oblikovanje konačnog modela. Budući da svaki koeficijent w_i množi pripadno svojstvo x_i i tako određuje utjecaj svojstva x_i na predikciju, vektor $\mathbf{w}$ se može promatrati kao set težina.

Npr. ako se svojstvu x_i pridruži pozitivna težina w_i svako povećanje ovog svojstva će rezultirati povećanjem vrijednosti predikcije $\hat{y}$. Također vrijedi, ako je pridružena težina negativna, svako povećanje vrijednosti svojstva će rezultirati smanjenjem vrijednosti predikcije $\hat{y}$. Dakle, što je pridružena težina w_i po vrijednosti veća to je i veći utjecaj svojstva x_i na predikciju $\hat{y}$. Samim tim, ako je težina jednaka nuli, svojstvo neće imati utjecaja na predikciju.

Neka $X^{(test)}$ sadrži m primjera ulaznih svojstava, a $y^{(test)}$ stvarne izlazne vrijednosti. Ako $\hat{y}^{(test)}$ sadrži predikcije modela za ulaz $X^{(test)}$ tada se može napraviti evaluacija modela izračunom srednje kvadratne vrijednosti greške (eng. Mean Squared Error, MSE) tako da vrijedi

$$MSE_{test} = \frac{1}{m} \sum_i (\hat{y}^{(test)} - y^{(test)})_i^2. \quad (11)$$

Lako uočljivo je da je evaluacija najbolja odnosno greška pada na nulu ako vrijedi $\hat{y}^{(test)} = y^{(test)}$.

Osim toga, može se prepoznati da rastom Euklidove udaljenosti između predikcije i stvarne vrijednosti raste i greška.

$$MSE_{test} = \frac{1}{m} \sum_i ||\hat{y}^{(test)} - y^{(test)}||_2^2 \quad (12)$$

Linearna regresija funkcionira na način da kroz trening podatke pokušava što bolje namjestiti težine za koje će grešku svesti na minimum, odnosno dobiti gradijent greške jednak nuli. To se matematički može opisati na sljedeći način,

$$\begin{aligned} \nabla_w MSE_{train} &= 0 \\ \Rightarrow \nabla_w \frac{1}{m} ||\hat{y}^{(train)} - y^{(train)}||_2^2 &= 0 \\ \Rightarrow \frac{1}{m} \nabla_w ||X^{(train)} w - y^{(train)}||_2^2 &= 0 \\ \Rightarrow \nabla_w (X^{(train)} w - y^{(train)})^T (X^{(train)} w - y^{(train)}) &= 0 \quad (13) \\ \Rightarrow \nabla_w (w^T X^{(train)T} X^{(train)} w - 2w^T X^{(train)T} y^{(train)} + y^{(train)T} y^{(train)}) &= 0 \\ \Rightarrow 2X^{(train)T} X^{(train)} w - 2X^{(train)T} y^{(train)} &= 0 \\ \Rightarrow w &= (X^{(train)T} X^{(train)})^{-1} X^{(train)T} y^{(train)} \end{aligned}$$

Potpuna jednačba modela linearne regresije glasi

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b \quad (14)$$

gdje oznaka „b“ označava odsječak pravca na osi y ili baznu (osnovnu) vrijednost. Npr. neka model predviđa plaću radnika ($\hat{y}$) na temelju duljine radnog staža ($\mathbf{x}$). Termin „b“ bi u tom slučaju predstavljao početnu plaću za radnike bez iskustva. [9]

3.1.2 Slučajna šuma

Kao što je već rečeno, slučajna šuma pripada metodama pakiranja koja je dio ansambl metoda. Naziv algoritma „slučajna šuma“ dolazi iz činjenice da se algoritam temelji na brojnim stablima odluke. Jedan od razloga nastanka ovog algoritma je sprječavanje prekomjernog učenja modela. Pojedinačna stabla se relativno lako prilagode karakteristikama pojedinih podataka umjesto da ih generaliziraju što nije slučaj kod slučajne šume. Za rješavanje klasifikacijskih problema svako stablo će dati svoj glas, a konačna odluka se donosi većinskim prebrojavanjem glasova. S druge strane, regresijski problem se rješava prosječnom vrijednošću svih stabala.

Algoritam slučajne šume započinje kreiranjem višestrukih stabala odluke. Svako stablo je izgrađeno na podskupu podataka za treniranje, izabranih kroz proces zvan „bootstrapping“. Ovaj proces se sastoji od nasumičnog odabiranja N uzoraka s ponavljanjem što znači da se neki uzorak može ponavljati više puta ili ne mora uopće biti odabran. Algoritam je detaljnije opisan pseudokodom na sljedeći način:

1. Za $b = 1$ do B , gdje je B broj stabala
 - a) Odaberi „bootstrap“ uzorak Z^* , veličine N iz skupa trening podataka
 - b) Izgradi stablo slučajne šume T_b trenirano na „bootstrap“ uzorku, rekursivno ponavljajući sljedeće korake za svaki terminalni čvor u stablu sve dok se ne postigne minimalna veličina čvora n_{min}
 - i. Od p mogućih izaberi m slučajnih varijabli
 - ii. Izaberi najbolju varijablu/točku podjele među odabranim m
 - iii. Podijeli čvor na lijevi i desni čvor
2. Spremi skup stabala $\{T_b\}_1^B$
3. Predikcija za novu točku x glasi:
 - a. Za regresiju: $\hat{f}_{rf}^B(x) = \frac{1}{B} \sum_{b=1}^B T_b(x)$
 - b. Za klasifikaciju: $\hat{C}_{rf}^B(x) = \text{najviše glasova} \{\hat{C}_b(x)\}_1^B$,
gdje je $\hat{C}_b(x)$ predikcija pojedinog stabla

Za B nasumičnih varijabli (stabala odluke) gdje pojedinačna varijabla ima varijancu σ^2 njihova prosječna varijanca će iznositi $\frac{1}{B}\sigma^2$. Također vrijedi, ako su varijable jednako distribuirane i među njima postoji pozitivna korelacija ρ tada prosječna varijanca glasi

$$\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2. \quad (15)$$

Iz formule se može zaključiti kako broj stabala B raste, drugi član se smanjuje te se tako i ukupna varijanca smanjuje. No, ipak smanjenje varijance je ograničeno jer prvi član ne ovisi o broju stabala. Ideja algoritma slučajne šume je reducirati varijancu tako što će smanjiti korelaciju između stabala. To se postiže izgradnjom stabala s nasumično odabranim ulaznim varijablama m . Tipična vrijednost za m iznosi $\sqrt{p}$. Nakon što je izgrađeno B stabala $\{T(x; \theta_b)\}_1^B$, predikcija regresijske slučajne šume glasi

$$\hat{f}_{rf}^B(x) = \frac{1}{B} \sum_{b=1}^B T(x; \theta_b), \quad (16)$$

gdje je θ_b b -to stablo iz slučajne šume. [10]

3.1.3 Poboljšano gradijentno pojačanje

Kao i slučajna šuma poboljšano gradijentno pojačanje također pripada ansambl metodama, ali drugoj podskupini odnosno metodama „pojačavanja“. Ovaj algoritam je zapravo optimizirana verzija „Gradient Boostinga“. Matematički se može objasniti na sljedeći način.

Neka se model sastoji od K stabala odluke, predikcija za prvih t stabala se može zapisati kao

$$\hat{y}_i^{(t)} = \sum_{k=1}^t f_k(x_i) \quad (17)$$

gdje je f_k predikcija svakog zasebnog stabla, a x_i ulazni podatci za i -ti redak tablice. Za treniranje modela potrebno je optimizirati funkciju gubitka (eng. Loss Function) te se za regresiju obično koristi srednja kvadratna pogreška

$$L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2. \quad (18)$$

Regularizacija je još jedan bitan dio modela jer kontrolira njegovu kompleksnost i tako sprječava prekomjerno treniranje, a može se definirati kao

$$\Omega = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2, \quad (19)$$

Gdje je T broj listova stabla, a w_j^2 je rezultat predikcije na j -tom listu. Ako se funkcija gubitka i regularizacija spoje u jednu funkcije, dobije se ciljna funkcija modela

$$Obj = L + \Omega \quad (20)$$

gdje funkcija gubitka osigurava preciznost modela, a regularizacija njegovu jednostavnost. Za optimizaciju funkcije cilja koristi se gradijentni spust. Gradijentni spust je iterativna tehnika koja će za funkciju cilja $Obj(y, \hat{y})$ izračunati

$$\partial_{\hat{y}} Obj(y, \hat{y}) \quad (21)$$

pri svakoj iteraciji. Zatim se unaprjeđuje $\hat{y}$ u smjeru gradijentnog spusta kako bi se minimizirala funkcija cilja. U svrhu iterativne optimizacije funkcija cilja se može zapisati kao

$$Obj^{(t)} = \sum_{i=1}^N L(y_i, \hat{y}_i^{(t)}) + \sum_{i=1}^t \Omega(f_i) = \sum_{i=1}^N L(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \sum_{i=1}^t \Omega(f_i). \quad (22)$$

Za optimizaciju gradijentnim spustom potrebno je izračunati gradijent. Optimizacija se također može poboljšati izračunom gradijenta prvog i drugog reda.

$$\partial_{\hat{y}_i^{(t)}} Obj^{(t)} \quad (23)$$

$$\partial_{\hat{y}_i^{(t)}}^2 Obj^{(t)} \quad (24)$$

Radi lakšeg računanja funkcija cilja se može zapisati aproksimacijom drugog Taylorovog reda,

$$Obj^{(t)} \simeq \sum_{i=1}^N \left[L(y_i, \hat{y}_i^{(t-1)}) + g_i f_i(x_i) + \frac{1}{2} h_i f_i^2(x_i) \right] + \sum_{i=1}^t \Omega(f_i) \quad (25)$$

gdje je

- $g_i = \partial_{\hat{y}_i^{(t-1)}} l(y_i, \hat{y}_i^{(t-1)})$
- $h_i = \partial_{\hat{y}_i^{(t-1)}}^2 l(y_i, \hat{y}_i^{(t-1)})$.

Ako se zanemare konstante dobije se pojednostavljeni oblik

$$Obj^{(t)} \simeq \sum_{i=1}^N \left[g_i f_i(x_i) + \frac{1}{2} h_i f_i^2(x_i) \right] + \Omega(f_t). \quad (26)$$

Cilj je pri svakoj iteraciji pronaći najoptimalniju funkciju f_t (stablo odluke) koja će minimizirati funkciju cilja.

Potrebno je kreirati stablo koje će unaprjeđivati predikciju prema gradijentnom spustu. Takvo stablo se može zapisati kao

$$f_t(x) = w_{q(x)} \quad (27)$$

gdje je $q(x)$ funkcija usmjeravanja koja mapira ulazne podatke s listom u stablu. Npr. $q(x)$ označava list u koji će završiti ulazni podatak x nakon svih podjela.

Neka set indeksa

$$I_j = \{i | q(x_i) = j\} \quad (28)$$

sadrži indekse ulaznih podataka koji su pridruženi j -tom listu stabla. Tada se funkcija cilja može zapisati kao

$$Obj^{(t)} = \sum_{i=1}^n \left[g_i f_i(x_i) + \frac{1}{2} h_i f_i^2(x_i) \right] + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (29)$$

$$= \sum_{j=1}^T \left[\left(\sum_{i \in I_j} g_i \right) w_j + \frac{1}{2} \left(\sum_{i \in I_j} h_i + \lambda \right) w_j^2 \right] + \gamma T.$$

Ovakav zapis sumira predikcije po listovima.

Za najbolji w_j

$$w_j = - \frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda} \quad (30)$$

gdje su g_i i h_i prvi i drugi red derivacije funkcije gubitka, a λ parametar regulacije, vrijedi

$$Obj^{(t)} = - \frac{1}{2} \sum_{j=1}^T \frac{\left(\sum_{i \in I_j} g_i \right)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T. \quad (31)$$

Cilj algoritma je pronaći najbolje ulazne podatke za koje će podjela trenutnog čvora na nova dva lista rezultirati unaprjeđenjem ciljne funkcije. Neka je I set indeksa ulaznih podataka koji su pridruženi trenutnom čvoru, a I_L i I_R setovi indeksa ulaznih podataka pridruženih novim listovima. Tada, najbolja vrijednost ciljne funkcije na j -tom listu iznosi

$$Obj^{(t)} = - \frac{\left(\sum_{i \in I_j} g_i \right)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma. \quad (32)$$

Podjelom čvora, računa se dobit podjele, tj. „gain“. Ova mjera govori koliko se ciljna funkcija unaprjeđuje u odnosu na zadnji čvor. Ukoliko je „gain“ pozitivan, podjela je profitabilna, inače vodi prekomjernom učenju. Formula za „gain“ se može zapisati kao

$$gain = \frac{1}{2} \left[\frac{\left(\sum_{i \in I_L} g_i \right)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{\left(\sum_{i \in I_R} g_i \right)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{\left(\sum_{i \in I} g_i \right)^2}{\sum_{i \in I} h_i + \lambda} \right] - \gamma. \quad (33)$$

Stablo se gradi tako što se rekurzivno traži najbolja točka podjele sve dok se ne postigne maksimalna dubina stabla. Nakon toga se uklanjaju čvorovi s negativnim „gainom“ redom odozdo prema gore.

Dakle, algoritam se može sažeti:

- Iteriraj n puta da se izgradi n stabala
 - 1) Izgradi stablo do maksimalne dubine
 - a) Pronađi najbolju točku podjele
 - b) Pridruži predikcije listovima
 - 2) Ukloni čvorove s negativnim „gainom“ [11]

3.1.4 Dugoročno - kratkoročna memorija

Dugoročno-kratkoročna memorija ili poznatije LSTM je napredna verzija rekurentne neuronske mreže. Za razliku od standardne mreže koja posjeduje samo skrivena stanja (kratkoročna memorija) LSTM posjeduje i memorijske ćelije (dugoročna memorija) koje omogućavaju učenje dugoročnih ovisnosti. Memorijskom ćelijom upravljaju tri vrste vrata kao što su vrata zaborava, ulazna te izlazna vrata. Vrata odlučuju koja informacija će se izbrisati, dodati ili služiti kao izlaz memorije.

Vrata zaborava

Vrata zaborava uklanjaju informacije iz memorijske ćelije ukoliko zaključe da su takve informacije suvisle. U vrata zaborava ulaze dva ulaza, x_t (trenutni ulaz) i h_{t-1} (izlaz iz prethodne ćelije). Oba ulaza se množe s pripadnim težinskim matricama te im se pribraja pristranost. Ishod ove operacije ulazi u sigmoidalnu aktivacijsku funkciju σ koja rezultira izlazima u rasponu od 0 do 1. Ukoliko je za određenu informaciju izlaz jednak nuli informacija se briše, a za rezultat jednak jedinici informacija ostaje. Operacija koju izvide vrata zaborava može se opisati matematički na sljedeći način,

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (34)$$

gdje su W_f i b_f odgovarajuće matrice težina i pristranosti.

Ulazna vrata

Ulazna vrata odlučuju koje informacije će se dodati u memorijsku ćeliju. Prvo se ulazne informacije x_t i h_{t-1} reguliraju pomoću sigmoidalne funkcije. Regulacija kao izlaz daje vrijednosti

između 0 i 1. Nula označava da se informacija odbacuje, a jedinica da se prihvaća. Regulacija se može matematički opisati kao

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (35)$$

gdje je W_i matrica težina, a b_i pristranost ulaznih vrata.

U međuvremenu se za iste ulaze računa vektor kandidata $\hat{C}_t$ pomoću $\tanh$ funkcije s vrijednostima između -1 i 1. Matematički se može zapisati ako

$$\hat{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c) \quad (36)$$

gdje je W_c matrica težina za generiranje vektora kandidata, a b_c faktor pristranosti (indeks c označava proces ažuriranja stanja ćelije).

Međutim, vektor $\hat{C}_t$ ne sadrži informacije o značaju trenutnih ulaznih podataka (hoće li zadržati ili odbaciti podatke) te se stoga množi element po element s vektorom i_t . Kao rezultat dobiju se vrijednosti koje određuju koliko se novih informacija iz $\hat{C}_t$ dodaje u ćeliju. Također, na isti način se množi vektor f_t s prethodnim stanjem ćelije C_{t-1} čiji rezultat govori koje će se informacije zadržati, a koje odbaciti iz prethodnog stanja. Stoga, vrijedi

$$C_t = f_t \times C_{t-1} + i_t \times \hat{C}_t, \quad (37)$$

gdje C_t predstavlja konačno stanje ćelije dobiveno filtriranjem starih i novih informacija.

Izlazna vrata

Izlazna vrata izdvajaju informacije iz trenutnog stanja ćelije koje će biti korisne kao ulaz sljedećoj ćeliji. Najprije se pomoću $\tanh$ funkcije kreira vektor kandidata koji sadrži moguće vrijednosti za izlaz u rasponu od -1 do 1. Potom se primjenom sigmoidalne funkcije reguliraju ulazi h_{t-1} i x_t tako da se kao njeni izlazi dobiju vrijednosti između 0 i 1. Na kraju se te vrijednosti pomnože s vektorom kandidata te se kao rezultat dobije trenutno skriveno stanje h_t koje se proslijeđuje kao ulaz za sljedeću ćeliju. Ove operacije se matematički mogu opisati na sljedeći način

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o), \quad (38)$$

$$h_t = o_t \cdot \tanh(C_t) \quad (39)$$

gdje je o_t izlaz sigmoidalne funkcije s matricom težina W_o , a b_o je faktor pristranosti. [12][13]

3.2 Evaluacijske metrike

3.2.1 Srednja apsolutna pogreška

Srednja apsolutna pogreška (eng. Mean Absolute Error, MAE) mjeri prosječnu apsolutnu pogrešku između predviđenih i stvarnih vrijednosti. To znači da će izračunati apsolutnu vrijednost razlike između predviđenih i stvarnih vrijednosti i tako eliminirati negativne vrijednosti. Zatim će kao rezultat uzeti prosječnu vrijednost među prethodno izračunatim apsolutnim vrijednostima. Matematički to se može zapisati kao

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (40)$$

gdje je y_i stvarna vrijednost, $\hat{y}_i$ predviđena vrijednost, a n broj uzoraka.

3.2.2 Srednja kvadratna pogreška

Srednja kvadratna pogreška (eng. Mean Squared Error, MSE) mjeri srednju kvadratnu pogrešku između predviđenih i stvarnih vrijednosti. Dakle, za razliku od MAE-a, MSE računa kvadrat pogreške te će tako veće pogreške imati veći utjecaj na ukupnu vrijednost MSE-a. MSE je jedna od popularnijih metrika za optimizaciju modela, a njen matematički zapis smo već vidjeli u potpoglavlju o linearnoj regresiji (11).

3.2.3 Korijen srednje kvadratne pogreške

Korijen srednje kvadratne pogreške (eng. Root Mean Squared Error, RMSE) je u suštini ista metrika kao i MSE, osim što na kraju računa drugi korijen iz ukupne vrijednosti. To olakšava interpretaciju pogreške. Npr. ako se predviđa cijena automobila u eurima, MSE će izračunati pogrešku kao kvadrat nekoliko tisuća eura. Izračunom korijena pogreška će biti u istim jedinicama kao i predviđene/stvarne vrijednosti. Matematički zapis glasi

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (41)$$

gdje je y_i stvarna vrijednost, $\hat{y}_i$ predviđena vrijednost, a n broj uzoraka.

3.2.4 Koeficijent determinacije

Koeficijent determinacije (eng. R-squared, R^2) je metrika koja objašnjava varijabilnost podataka, odnosno koliko dobro model objašnjava varijabilnost zavisne varijable na temelju nezavisnih. Vrijednost R^2 se kreće između 0 i 1 te vrijedi što je rezultat bliži 1 to model bolje objašnjava varijabilnost. Formula za izračun glasi

$$R^2 = 1 - \frac{SSR}{SST} \quad (42)$$

gdje SSR predstavlja zbroj kvadrata reziduala između predviđenih i stvarnih vrijednosti, a SST ukupni zbroj kvadrata koji mjeri ukupnu varijaciju ovisne varijable. [14]

4 PRAKTIČNI DIO RADA

Ovo poglavlje se bavi programskom implementacijom rada. Poglavlje započinje navođenjem korištenih tehnologija za izradu rada, a potom se navode detalji oko korištenog skupa podataka poput pred-procesiranja i same pripreme za provođenje eksperimenta. Nakon toga se opisuje proces treniranja različitih modela i njihovih rezultata. Na kraju poglavlja uspoređeni su međusobni rezultati svih modela.

4.1 Korištene tehnologije

Kao programsko okruženje za izradu praktičnog dijela korišten je Google Colab. Google Colab je platforma za izvršavanje Python koda u oblaku. Temelji se na Jupyter bilježnicama pa nisu potrebna nikakva prethodna podešavanja okruženja. Također, pruža besplatne računalne resurse poput CPU-a, GPU-a, TPU-a, RAM-a, diskovnog prostora za pohranu, itd. Naravno za besplatni plan korištenja resursi su ograničeni. Konkretno, za implementaciju ovoga rada korišten je Python 3.10, a kao dostupni resursi na raspolaganju su bile dvije virtualne procesorske jezgre sa snagom od 2.2 GHz svaka posebno, T4 GPU, TPU v2, 12.7 GB RAM-a i 78 GB prostora za pohranu. [15]

Za potrebe rada korištene su brojne biblioteke kao što su yfinance, numpy, pandas, scikit-learn, tensorflow, keras, xgboost, matplotlib i seaborn. Yfinance je biblioteka koja omogućava dohvaćanje povijesnih cijena dionica, indeksa ili kripto valuta s Yahoo Finance servisa. Numpy i pandas biblioteke pružaju mogućnost jednostavne analize i obrade podataka. Scikit-learn biblioteka omogućava korištenje algoritama linearne regresije i slučajne šume. Osim toga sadrži brojne druge mogućnosti poput normalizacije podataka ili evaluacijskih metrika. Tensorflow i keras su biblioteke koje se koriste za kreiranje i učenje dubokih neuronskih mreža, a xgboost za kreiranje modela poboljšanog gradijentnog učenja. Matplotlib i seaborn biblioteke koriste se za vizualizaciju podataka. [16][17]

4.2 Analiza podataka

Analiza podataka je jedan od ključnih procesa unutar implementacije strojnog učenja. Jedan od podkoraka je istraživačka analiza podataka (eng. Exploratory Data Analysis, EDA) koja uključuje otkrivanje odnosa unutar podataka grafičkim metodama što ćemo vidjeti u nastavku.

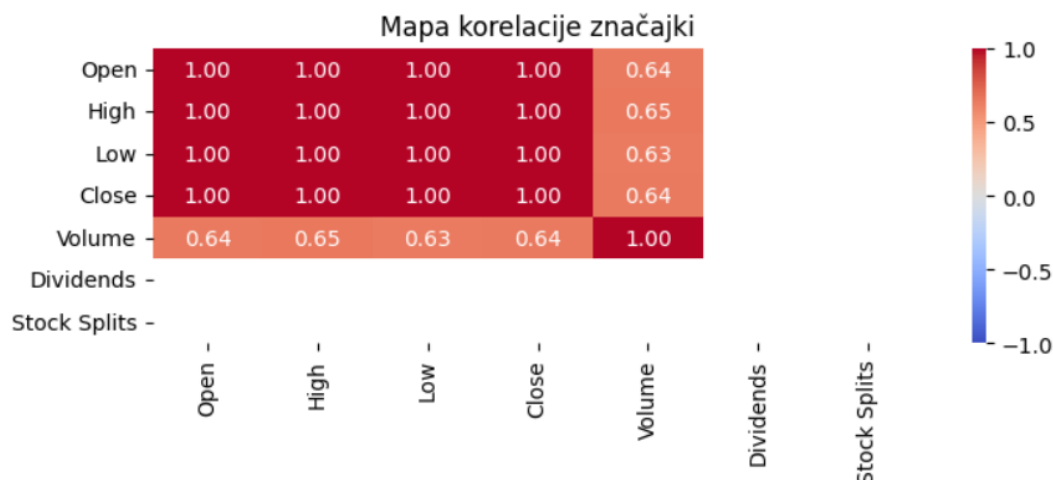
Korišteni set podataka preuzet je s internetske platforme „Yahoo! Finance“ koja nudi široki spektar informacija za dionice, kripto valute i slično. Dohvaćeni podaci o bitcoinu su aktualni s vremenom, a datiraju od 17.09.2014. godine pa sve do današnjega dana. Set podataka sadrži više od 3600 uzoraka za 7 različitih značajki kao što su „Open“, „High“, „Low“, „Close“, „Volume“, „Dividends“ i „Stock Splits“. Sve informacije o bitcoinu su prikazane u odnosu na američki dolar. Slika 4-1 prikazuje prvih pet uzoraka iz danog seta podataka. [18]

	Open	High	Low	Close	Volume	Dividends	Stock Splits
Date							
2014-09-17 00:00:00+00:00	465.864014	468.174011	452.421997	457.334015	21056800	0.0	0.0
2014-09-18 00:00:00+00:00	456.859985	456.859985	413.104004	424.440002	34483200	0.0	0.0
2014-09-19 00:00:00+00:00	424.102997	427.834991	384.532013	394.795990	37919700	0.0	0.0
2014-09-20 00:00:00+00:00	394.673004	423.295990	389.882996	408.903992	36863600	0.0	0.0
2014-09-21 00:00:00+00:00	408.084991	412.425995	393.181000	398.821014	26580100	0.0	0.0

Slika 4-1 Set podataka

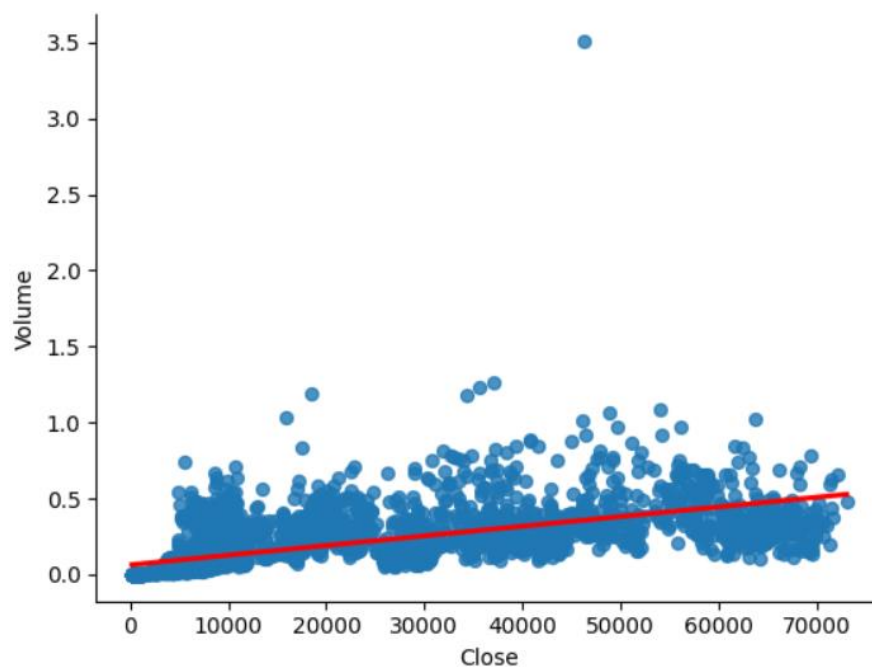
Analizom podataka i pregledom korelacijske mape može se uvidjeti da značajke „Dividends“ i „Stock Splits“ nemaju korelaciju s ostalim značajkama te će iste biti uklonjene kroz proces pripreme podataka.

Slika 4-2 prikazuje već navedeno, ali i slabiju korelaciju značajke „Volume“ s ostalima.



Slika 4-2 Korelacija značajki

Kao ciljna, odabrana je „Close“ značajka koja predstavlja zaključnu cijenu bitcoina za određeni datum. Na ostalima će se obavljati trening modela gdje „Open“ predstavlja početnu cijenu, „High“ najvišu, a „Low“ najnižu za već spomenuti datum. „Volume“ predstavlja ukupan protok novca nastao kupnjom i prodajom bitcoina na burzama. Slika 4-3 prikazuje odnos kretanja zaključne cijene naspram volumena, može se uočiti kako s porastom volumena i zaključne cijene su sve više.



Slika 4-3 Odnos volumena i zaključne cijene

4.3 Predobrada podataka i *feature engineering*

Predobrada podataka je korak unutar strojnog učenja koji se sastoji od čišćenja i transformacije podataka kako bi bili prikladni za modeliranje. *Feature engineering* je postupak stvaranja novih značajki iz postojećih što pomaže modelima u boljem razumijevanju relacija među podacima.

Kao što je već najavljeno u prethodnom potpoglavlju, u prvom koraku predobrade podataka uklonjeni su stupci sa značajkama „Dividends“ i „Stock Splits“. Također, indeksi redaka su konvertirani u „datetime format“ i uklonjeni su podaci o vremenskim zonama radi lakšeg korištenja. Slika 4-4 prikazuje dio seta podataka nakon prvog koraka predobrade podataka.

	Open	High	Low	Close	Volume
Date					
2014-09-17	465.864014	468.174011	452.421997	457.334015	21056800
2014-09-18	456.859985	456.859985	413.104004	424.440002	34483200
2014-09-19	424.102997	427.834991	384.532013	394.795990	37919700
2014-09-20	394.673004	423.295990	389.882996	408.903992	36863600
2014-09-21	408.084991	412.425995	393.181000	398.821014	26580100

Slika 4-4 Prvi korak predobrade podataka

Nadalje, kao dio *feature engineeringa* izračunati su klizni prosjeci značajki za vremenske horizonte od 2, 7, 60 i 365 dana. Korištenjem različitih horizonta, model može prepoznati različite tržišne cikluse i ponašanja. Horizonti od 2, 7, 60 i 365 dana su odabrani kako bi uhvatili raspon tržišnih trendova, kako kratkoročnih tako i dugoročnih. Konkretno, 2-dnevni horizont se koristi za hvatanje kratkoročnih dnevnih fluktuacija, a 7-dnevni za trendove na tjednoj bazi. 60-dnevni period je srednjoročni period koji hvata trendove u rasponu od 2 mjeseca, a takve možda neće prepoznati kratkoročni trendovi. Horizont od 365 dana se koristi za praćenje fluktuacija na godišnjoj razini. Nakon izračuna kliznih prosjeka, kreiraju se četiri nove značajke dobivene kao omjer „Open“ značajke i njenog kliznog prosjeka za svaki vremenski horizont. Dodatno, kreirana su četiri trenda za „Open“ značajku. Na kraju su odbačeni retci kojima nedostaju vrijednosti.

Kao posljednji korak predobrade podataka vrijednosti su skalirane primjenom `MinMaxScaler()` funkcije iz `scikit-learn` biblioteke. `MinMaxScaler()` provodi skaliranje tako da transformira vrijednosti značajki u rasponu od 0 do 1. Skaliranje je provedeno iz razloga jer su vrijednosti nekih značajki poput „Volume“ dosezale više milijardi, a značajke nastale *feature engineeringom* oko 1. Nakon skaliranja set podataka je podijeljen na trening i test skupove podataka u omjeru 80:20.

Slika 4-5 prikazuje nekoliko uzoraka testnog skupa podataka nakon provedene predobrade podataka i *feature engineeringa*.

	Open	High	Low	Volume	open_ratio_2	trend_2	open_ratio_7	trend_7	open_ratio_60	trend_60	open_ratio_365	trend_365
0	0.003171	0.002729	0.003115	0.000059	0.717845	0.003002	0.434082	0.003134	0.266006	0.002660	0.111924	0.002559
1	0.003048	0.002654	0.003170	0.000052	0.634801	0.002891	0.419064	0.003073	0.257294	0.002653	0.108212	0.002692
2	0.003089	0.003125	0.003157	0.000112	0.679146	0.002922	0.435920	0.003011	0.262452	0.002616	0.109397	0.002677
3	0.003551	0.003051	0.003508	0.000070	0.787845	0.002880	0.532666	0.002977	0.300348	0.002603	0.121243	0.002684
4	0.003378	0.002880	0.003344	0.000060	0.624262	0.003134	0.508515	0.003021	0.281280	0.002678	0.114492	0.002821

Slika 4-5 Testni skup podataka

4.4 Trening i rezultati linearne regresije

Linearna regresija je relativno jednostavan model te nema veliki broj hiperparametara za podešavanje. Konkretno za ovaj model korišten je samo „fit_intercept“ hiperparametar koji određuje hoće li model računati baznu vrijednost (pristranost) prilikom treninga.

Nadalje, u svrhu postizanja boljih rezultata korištena je tehnika „cross-validation“. Klasične „cross-validation“ metode nisu prikladne jer se radi o podacima koji imaju vremenski poredak pa bi njihova primjena narušila strukturu podataka. U ovom slučaju korištena je `TimeSeriesSplit()` funkcija koja dijeli skup podataka tri puta na trening i validacijski skup, ali s očuvanjem vremenskog poretka podataka.

Kao evaluacijska funkcija za pronalazak najboljih hiperparametara korištena je negativna srednja kvadratna pogreška. Razlog tomu je što `scikit-learn` funkcije koriste princip maksimizacije evaluacijske funkcije, a MSE je metrika koju je potrebno minimizirati (manja vrijednost = bolji rezultat).

```
parameters = {
    'fit_intercept': [True, False]
```

```

}

lr_model = LinearRegression()
tscv = TimeSeriesSplit(n_splits=3)

random_search = RandomizedSearchCV(estimator=lr_model,
param_distributions=parameters, cv=tscv, verbose=1, n_jobs=-1,
scoring='neg_mean_squared_error', random_state=1)

```

Naime, prilikom traženja najboljih hiperparametara utvrđeno je da model daje bolje rezultate kada računa baznu vrijednost. Slika 4-6 prikazuje rezultate evaluacijskih metrika nakon treniranja modela. Prema ovim podacima, srednje odstupanje predviđene od stvarne cijene iznosi otprilike 263.05 američkih dolara.

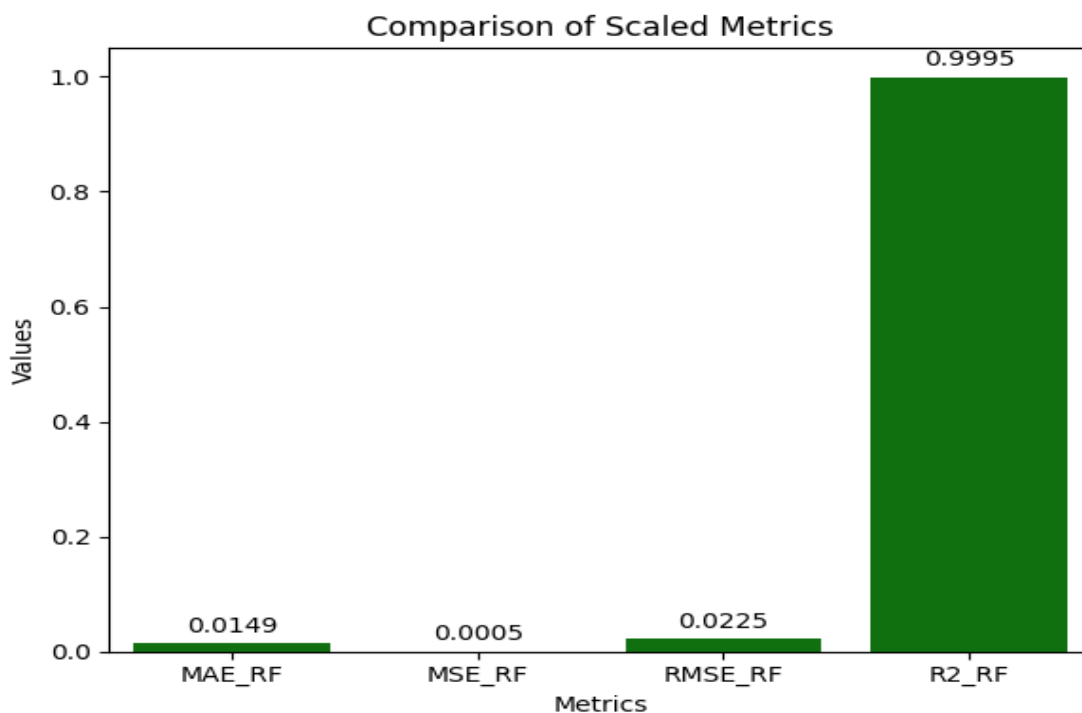
```

LR - Mean Absolute Error: 263.0539995770026
LR - Mean Squared Error: 157412.6011451398
LR - Root Mean Squared Error: 396.7525691727022
LR - R^2 Score: 0.9994954019648169

```

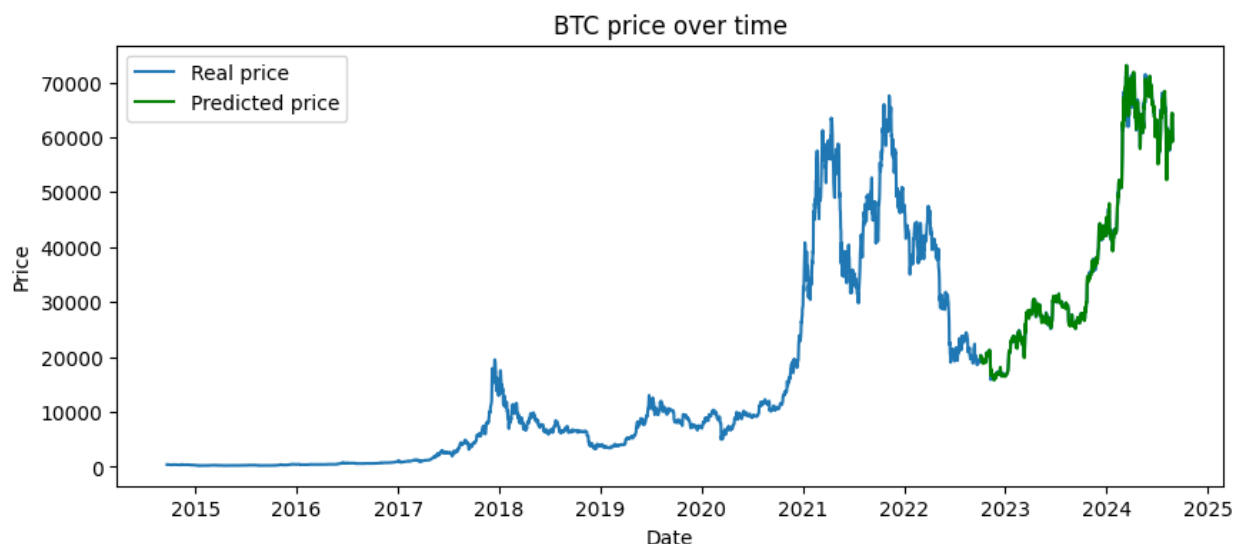
Slika 4-6 Rezultati evaluacijskih metrika – LR

Slika 4-7 prikazuje skalirane rezultate evaluacijskih metrika.



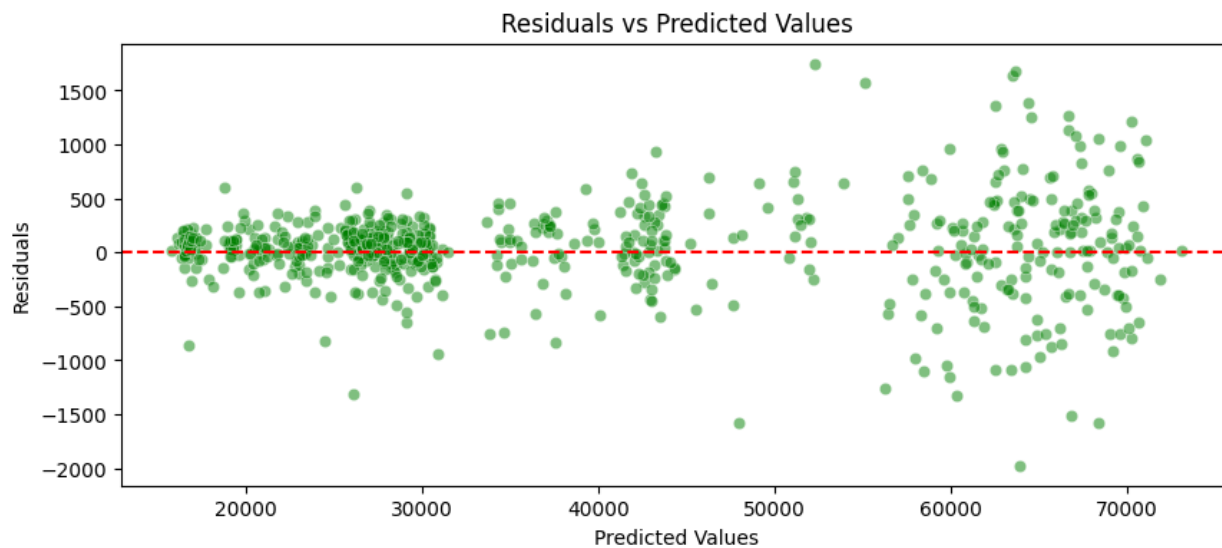
Slika 4-7 Skalirani rezultati evaluacijskih metrika (graf) – LR

Slika 4-8 prikazuje graf na kojemu su uspoređene stvarne i predviđene „Close“ cijene kroz vrijeme. Plavom bojom su prikazane stvarne, a zelenom predviđene vrijednosti.



Slika 4-8 Usporedba stvarnih i predviđenih cijena – LR

Slika 4-9 prikazuje reziduale, odnosno svaka točka prikazuje koliko je predviđena vrijednost odstupala od stvarne. Dakle, što je točka bliže vrijednosti 0 to je predviđanje bilo bolje.



Slika 4-9 Reziduali – LR

Slika 4-10 prikazuje usporedbu nekoliko prvih i zadnjih rezultata predviđanja gdje lijevi stupac označava stvarne, a desni predviđene vrijednosti.

	y_test	y_pred_lr
Date		
2022-10-04	20336.843750	20123.381528
2022-10-05	20160.716797	19864.591587
2022-10-06	19955.443359	20108.818902
2022-10-07	19546.849609	19572.545532
2022-10-08	19416.568359	19336.483210
...	...	...
2024-08-24	64178.992188	63794.575471
2024-08-25	64333.542969	64367.521969
2024-08-26	62880.660156	63226.981843
2024-08-27	59504.132812	59876.597668
2024-08-28	59653.402344	59356.041762

Slika 4-10 Stvarne i predviđene vrijednosti – LR

4.5 Trening i rezultati slučajne šume

Algoritam slučajne šume za razliku od linearne regresije ima veći spektar hiperparametara. Korišteni hiperparametri su „n_estimators“, „max_features“, „max_depth“, „min_samples_split“ i „min_samples_leaf“. „n_estimators“ označava broj stabala odluke unutar jedne šume (ansambla), a „max_features“ maksimalni broj značajki koje se koriste pri razmatranju podjele na svakom čvoru stabla. Nadalje, „max_depth“ predstavlja maksimalnu dubinu pojedinačnog stabla, „min_samples_split“ minimalni broj uzoraka potreban da se čvor podijeli, a „min_samples_leaf“ minimalni broj uzoraka koji mora biti prisutan u listovima stabla.

Kao i kod linearne regresije korištena je ista „cross-validation“ tehnika te evaluacijska funkcija.

```
parameters = {
    'n_estimators': [50, 100, 200, 300],
    'max_features': ['sqrt', 'log2', None],
    'max_depth': [5, 10, 15],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
```

```

}

rf = RandomForestRegressor(random_state=1)
tscv = TimeSeriesSplit(n_splits=3)

random_search = RandomizedSearchCV(estimator=rf,
param_distributions=parameters, n_iter=200, cv=tscv,
n_jobs=-1, verbose=1, scoring='neg_mean_squared_error', random_state=1)

```

Za traženje najboljeg modela slučajne šume isprobano je 200 različitih kombinacija hiperparametara. Model daje najbolje rezultate kada „n_estimators“ iznosi 50, „max_features“ ima vrijednost „None“, „max_depth“ vrijednost 10, „min_samples_split“ vrijednost 5 te konačno „min_samples_leaf“ je jednak 1. Slika 4-11 prikazuje rezultate evaluacijskih metrika za model slučajne šume. Srednje odstupanje predviđene cijene od stvarne iznosi 716.08 američkih dolara.

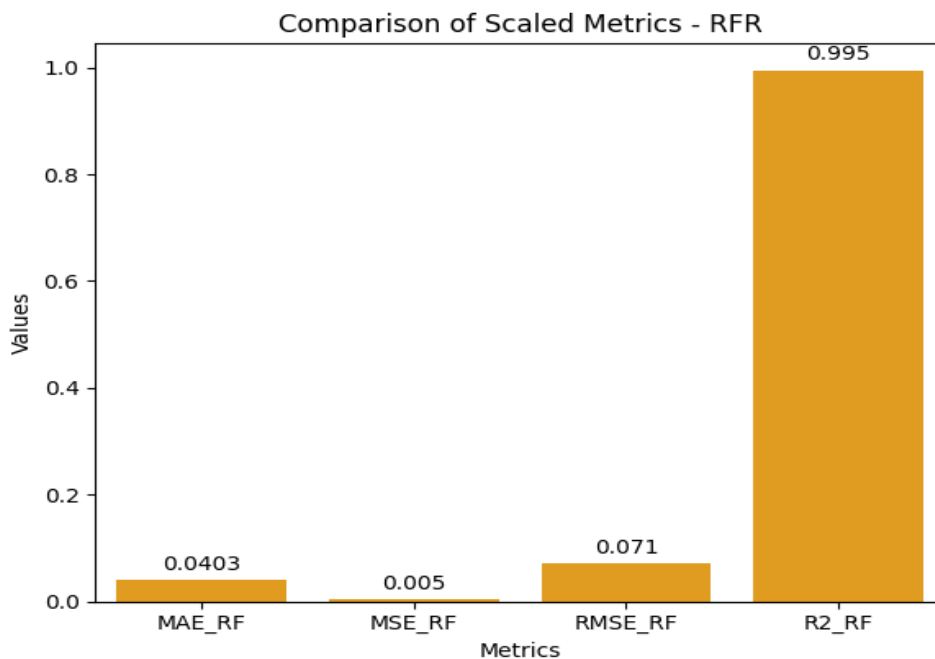
```

RFR - Mean Absolute Error: 712.652699947329
RFR - Mean Squared Error: 1571608.0802969404
RFR - Root Mean Squared Error: 1253.6379382808022
RFR - R-squared: 0.9949620910675092

```

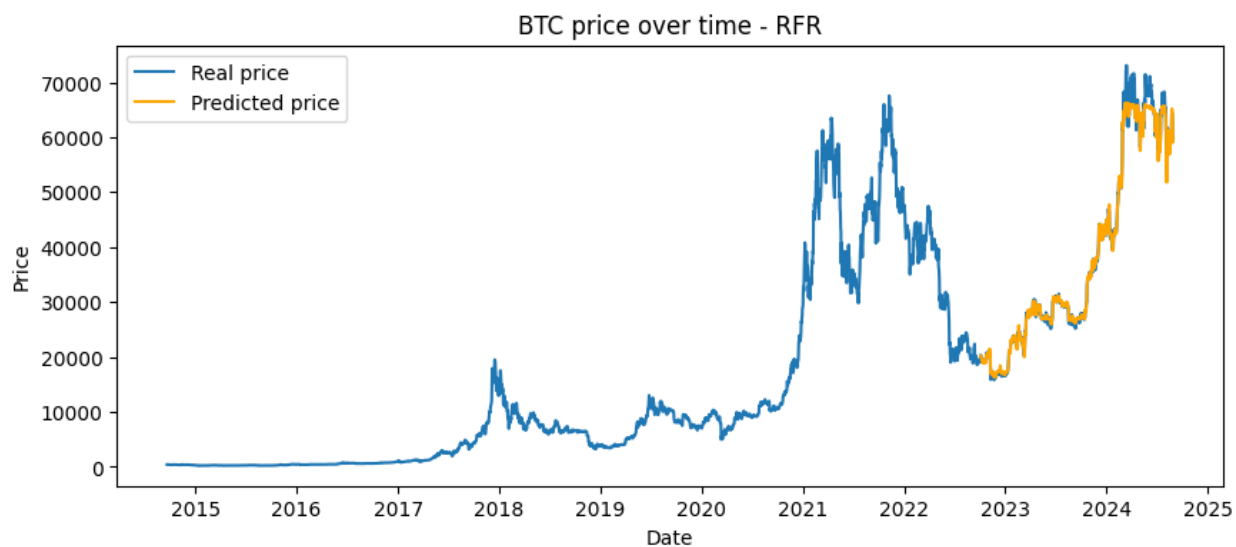
Slika 4-11 Rezultati evaluacijskih metrika – RFR

Slika 4-12 prikazuje skalirane rezultate evaluacijskih metrika.



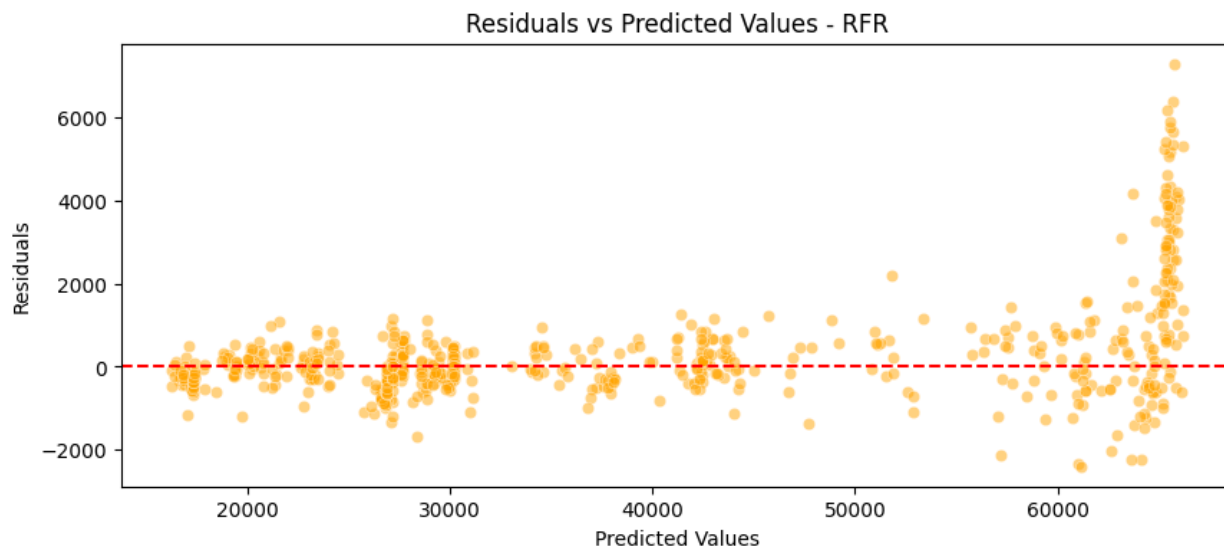
Slika 4-12 Skalirani rezultati evaluacijskih metrika (graf) - RFR

Slika 4-13 prikazuje graf na kojemu su uspoređene stvarne i predviđene „Close“ cijene kroz vrijeme. Plavom bojom su prikazane stvarne, a narančastom predviđene vrijednosti.



Slika 4-13 Usporedba stvarnih i predviđenih cijena – RFR

Slika 4-14 prikazuje reziduale za model slučajne šume.



Slika 4-14 Reziduali – RFR

Slika 4-15 prikazuje usporedbu nekoliko prvih i zadnjih rezultata predviđanja slučajne šume.

	y_test	y_pred_rf
Date		
2022-10-04	20336.843750	20144.660057
2022-10-05	20160.716797	20211.097950
2022-10-06	19955.443359	20101.655351
2022-10-07	19546.849609	19519.567008
2022-10-08	19416.568359	19423.865390
...	...	...
2024-08-24	64178.992188	65186.771804
2024-08-25	64333.542969	64753.911464
2024-08-26	62880.660156	64080.509139
2024-08-27	59504.132812	60738.935138
2024-08-28	59653.402344	59162.389855

Slika 4-15 Stvarne i predviđene vrijednosti – RFR

4.6 Trening i rezultati poboljšanog gradijentnog pojačanja

Poboljšano gradijentno pojačanje sadrži hiperparametre slične onima kod slučajne šume. Korišteni hiperparametri su „n_estimators“, „learning_rate“, „max_depth“, „subsample“ i „colsample_bytree“. „n_estimators“ ponovno označava broj stabala odluke unutar šume, „max_depth“ maksimalnu dubinu stabla, a „colsample_bytree“ određuje udio nasumičnih značajki koje će se koristiti za izgradnju pojedinačnog stabla. „learning_rate“ diktira veličinu koraka pri učenju, niža vrijednost može povećati točnost, ali zahtjeva više stabala (resursa). „subsample“ predstavlja nasumični odabir dijela uzoraka i tako pomaže spriječiti prekomjerno učenje.

Ponovno je korištena ista „cross-validation“ tehnika te evaluacijska funkcija.

```
parameters = {
    'n_estimators': [50, 100, 200, 500],
    'learning_rate': [0.01, 0.05, 0.1, 0.2],
    'max_depth': [3, 5, 7, 10],
    'subsample': [0.6, 0.8, 1.0],
```

```

    'colsample_bytree': [0.6, 0.8, 1.0]
}

xgb_model = xgb.XGBRegressor(objective='reg:squarederror')
tscv = TimeSeriesSplit(n_splits=3)

random_search = RandomizedSearchCV(estimator=xgb_model,
param_distributions=parameters, n_iter=200, cv=tscv, verbose=1,
scoring='neg_mean_squared_error', n_jobs=-1, random_state=1)

```

Među 200 različitih kombinacija hiperparametara, model je najbolje rezultirao kada „n_estimators“ iznosi 100, „learning_rate“ 0.1, „max_depth“ 3, „subsample“ 1.0 te „colsample_bytree“ 0.6. Slika 4-16 prikazuje rezultate evaluacijskih metrika. Vidljivo je da srednje odstupanje predviđene cijene od stvarne iznosi 976.87 američkih dolara.

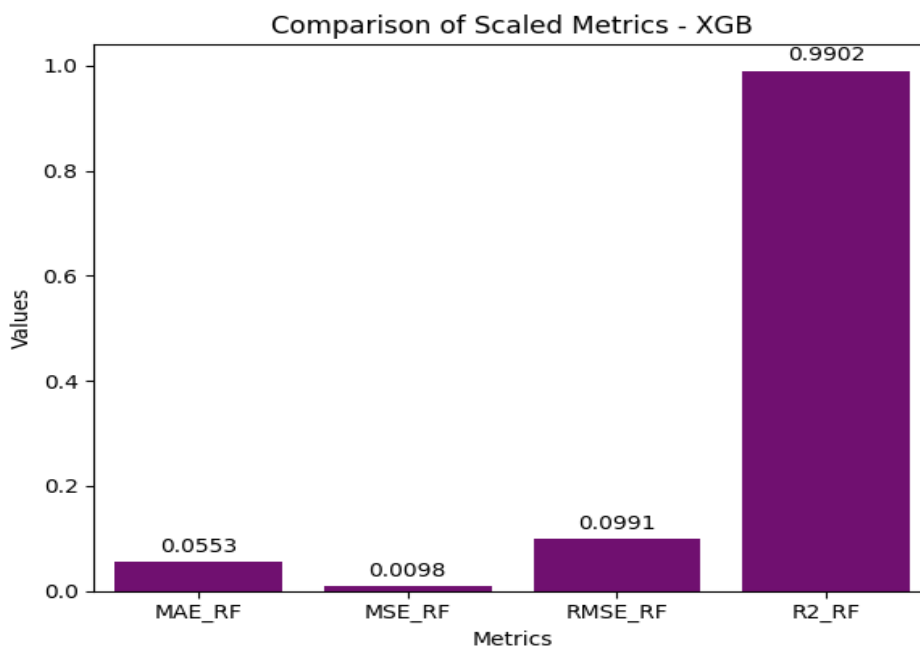
```

XGB - Mean Absolute Error: 976.8675089854511
XGB - Mean Squared Error: 3065974.071603138
XGB - Root Mean Squared Error: 1750.9923105494033
XGB - R^2 Score: 0.9901717875113011

```

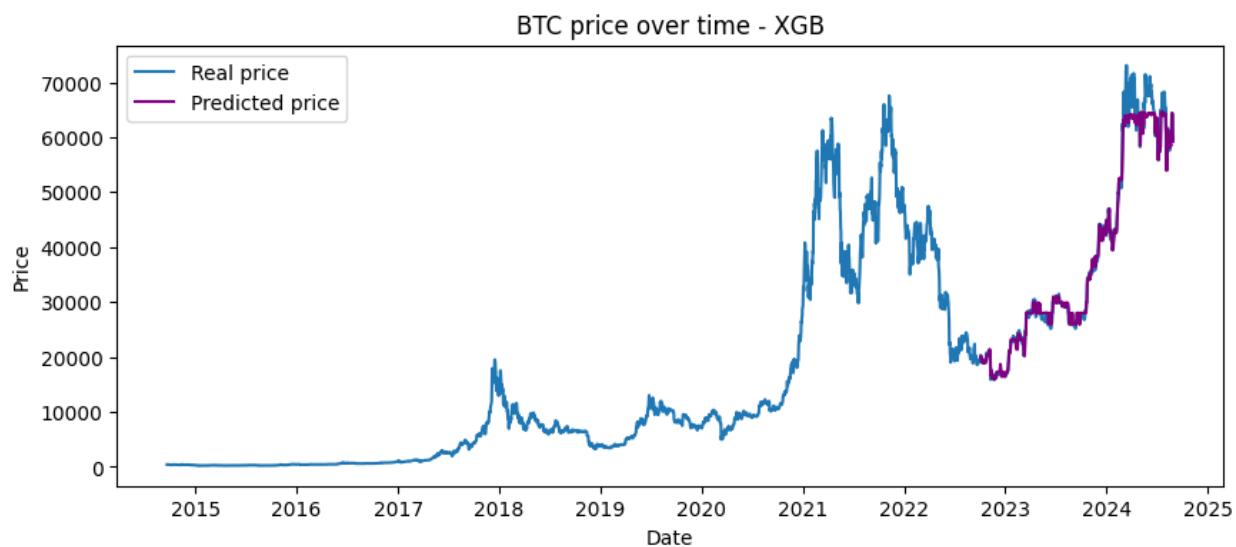
Slika 4-16 Rezultati evaluacijskih metrika – XGB

Slika 4-17 prikazuje skalirane rezultate evaluacijskih metrika.



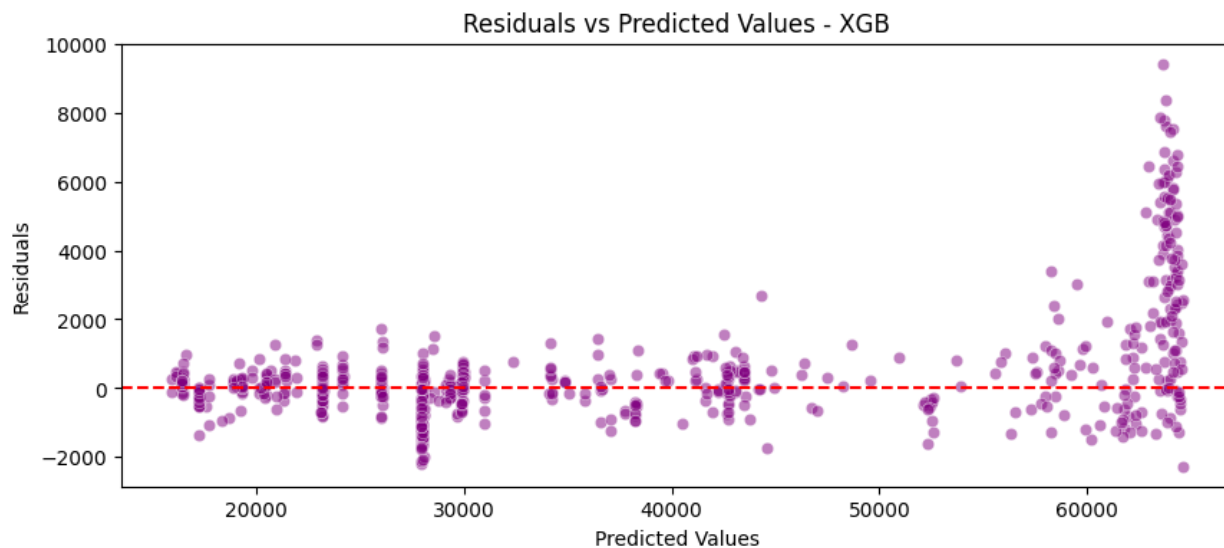
Slika 4-17 Skalirani rezultati evaluacijskih metrika (graf) – XGB

Slika 4-18 prikazuje graf na kojemu su uspoređene stvarne i predviđene „Close“ cijene kroz vrijeme. Plavom bojom su prikazane stvarne, a ljubičastom predviđene vrijednosti.



Slika 4-18 Usporedba stvarnih i predviđenih cijena – XGB

Slika 4-19 prikazuje reziduale za XGB model.



Slika 4-19 Reziduali – XGB

Slika 4-20 prikazuje usporedbu nekoliko prvih i zadnjih rezultata predviđanja XGB modela.

	y_test	y_pred_xgb
Date		
2022-10-04	20336.843750	19809.113281
2022-10-05	20160.716797	20124.437500
2022-10-06	19955.443359	20124.437500
2022-10-07	19546.849609	19305.009766
2022-10-08	19416.568359	19299.330078
...	...	...
2024-08-24	64178.992188	64401.679688
2024-08-25	64333.542969	64422.042969
2024-08-26	62880.660156	63868.652344
2024-08-27	59504.132812	60591.621094
2024-08-28	59653.402344	59245.492188

Slika 4-20 Stvarne i predviđene vrijednosti – XGB

4.7 Trening i rezultati dugoročno – kratkoročne memorije

Za potrebe treniranja LSTM algoritma ulazne podatke je potrebno dodatno transformirati, odnosno pretvoriti u 3D i 1D NumPy niz. Tada će set podataka koji se sastoji od značajki za treniranje (X_train) imati format [broj uzoraka, broj vremenskih koraka, broj značajki]. Broj uzoraka i broj značajki će kao što samo ime govori predstavljati broj uzoraka, odnosno značajki u novo formatiranom nizu. Dok broj vremenskih koraka označava koliko uzoraka iz prethodnog seta podataka će činiti jedan uzorak u novom nizu. Npr. ako broj vremenskih koraka iznosi 30 (mjesec dana), to znači da će se uzeti 30 uzoraka (dana) iz prethodnog seta podataka koji će predstavljati samo jedan uzorak u novom 3D nizu. Sljedeći uzorak 3D niza će činiti zadnjih 29 dana prethodnog uzorka + jedan novi dan, i tako redom. Na taj način model gleda 30 dana unatrag kako bi predvidio sljedeću vrijednost. Matrica ciljne varijable (y_train) se transformira tako da će kao nove vrijednosti uzimati svoje vrijednosti koje se mapiraju na zadnji dan uzorka u 3D nizu. Npr. ako se uzorak matrice X_train sastoji od prvih 30 dana uzoraka njemu će se kao ciljna vrijednost pridružiti

ciljna vrijednost 30. dana iz netransformirane matrice `y_train`, zatim idućem uzorku ciljna vrijednost 31. dana i tako redom.

```
def transform_dataset(X, y, time_step=30):
    X_3d, y_1d = [], []
    for i in range(len(X) - time_step):
        v = X[i:(i + time_step)]
        X_3d.append(v)
        y_1d.append(y[i + time_step - 1])
    return np.array(X_3d), np.array(y_1d)
```

Također, podaci su podijeljeni u novom omjeru 70:20:10 gdje se 70% podataka odnosi na trening, 20% na test, a 10% na validacijske podatke. Validacijski set se koristi kao set podataka koje model još nije vidio za ocjenu performansi modela tokom treniranja. Pomoću njih „loss“ funkcija omogućava modelu bolje postavljanje hiperparametara, generaliziranje podataka i sprječavanje prekomjernog učenja. Konkretno za ovaj model kao „loss“ funkcija korištena je srednja kvadratna pogreška.

Model je definiran kao sekvencijalni model s ukupno sedam slojeva. Sekvencijalni znači da se slojevi dodaju jedan za drugim linearnim slijedom. Prva tri sloja su LSTM slojevi koji imaju sposobnost pamćenja dugoročnih ovisnosti u podacima. Četvrti sloj je „Dropout“ sloj koji odbacuje određeni postotak neurona iz mreže kako bi spriječio prekomjerno učenje modela. Zadnja tri sloja su „Dense“ ili gusti slojevi što znači da su svi neuroni sloja povezani sa svim neuronima prethodnog sloja. Gusti slojevi su osnovni slojevi neuronskih mreža koji služe mreži da uče složenije obrasce.

```
def build_model(hp):
    model = Sequential()

    model.add(LSTM(units=hp.Int('units1', min_value=32, max_value=64, step=8),
        return_sequences=True, input_shape=(time_step, X_train.shape[2])))

    model.add(LSTM(units=hp.Int('units2', min_value=32, max_value=64, step=8),
        return_sequences=True))

    model.add(LSTM(units=hp.Int('units3', min_value=32, max_value=64, step=8)))
```

```

model.add(Dropout (hp.Float('dropout',      min_value=0.1,      max_value=0.3,
step=0.1)))

model.add(Dense(units=hp.Int('dense_units1',  min_value=16,  max_value=64,
step=8), activation="relu"))

model.add(Dense(units=hp.Int('dense_units2',  min_value=16,  max_value=64,
step=8), activation="relu"))

model.add(Dense(1))

model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=hp.Choice('
learning_rate', values=[1e-2, 1e-3, 1e-4])), loss='mean_squared_error')

return model

```

Među 50 različitih kombinacija hiperparametara model je najbolje rezultirao kada prvi lstm sloj ima 64, drugi 32 i treći 48 neurona. Također, najbolja kombinacija se sastoji od toga kada „dropout“ sloj odbacuje 20% neurona te kada prvi gusti sloj ima 32, odnosno drugi gusti sloj 64 neurona. Kao najbolji „learning_rate“ rezultira 0.001. Slika 4-21 prikazuje rezultate evaluacijskih metrika. Vidljivo je da srednje odstupanje predviđene cijene iznosi 637.86 američkih dolara.

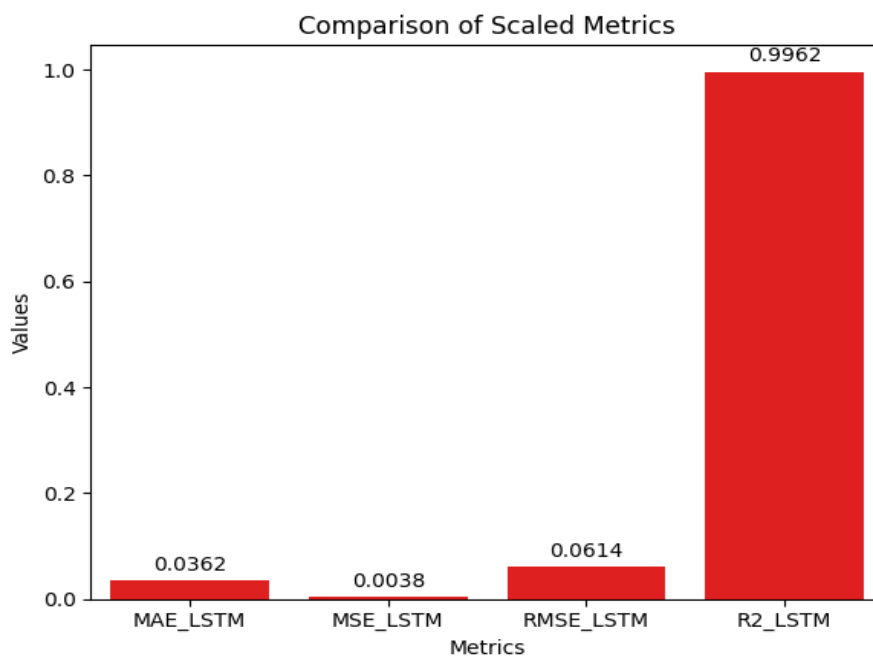
```

LSTM - Mean Absolute Error: 637.8586408267576
LSTM - Mean Squared Error: 1173075.8470405021
LSTM - Root Mean Squared Error: 1083.0862602029915
LSTM - R^2: 0.996229515497659

```

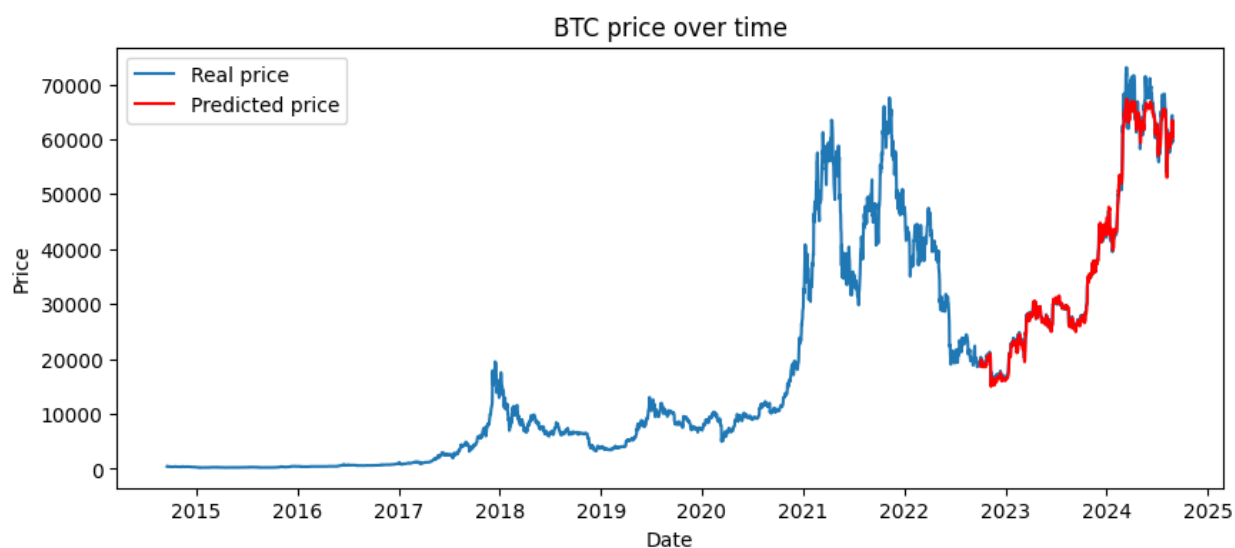
Slika 4-21 Rezultati evaluacijskih metrika –LSTM

Slika 4-22 prikazuje skalirane rezultate evaluacijskih metrika.



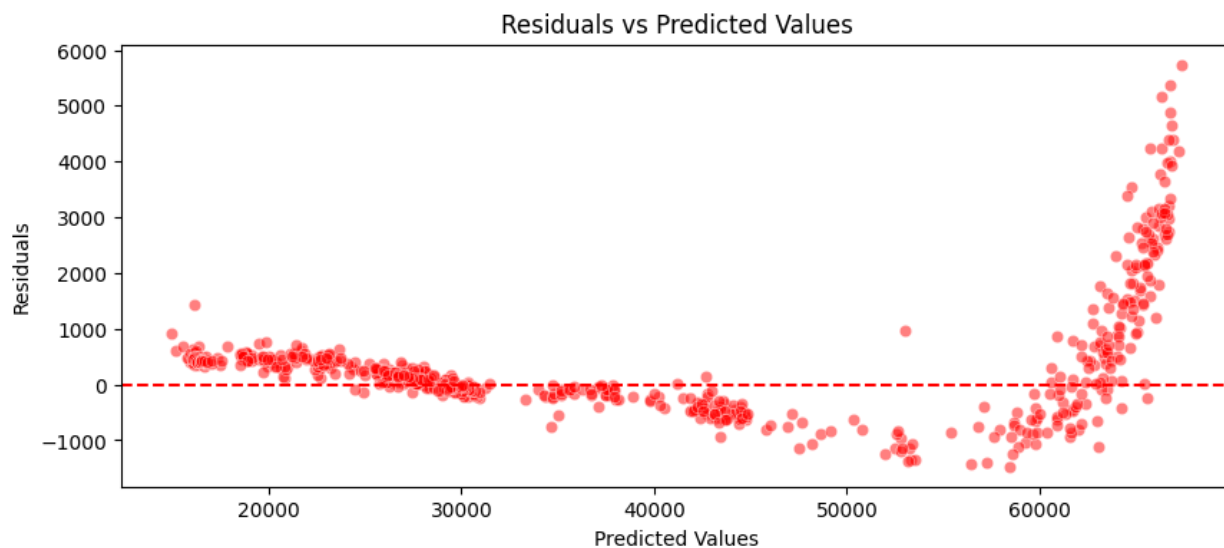
Slika 4-22 Skalirani rezultati evaluacijskih metrika (graf) – LSTM

Slika 4-23 prikazuje graf na kojemu su uspoređene stvarne i predviđene „Close“ cijene kroz vrijeme. Plavom bojom su prikazane stvarne, a crvenom predviđene vrijednosti.



Slika 4-23 Usporedba stvarnih i predviđenih cijena – LSTM

Slika 4-24 prikazuje reziduale za LSTM model.



Slika 4-24 Reziduali – LSTM

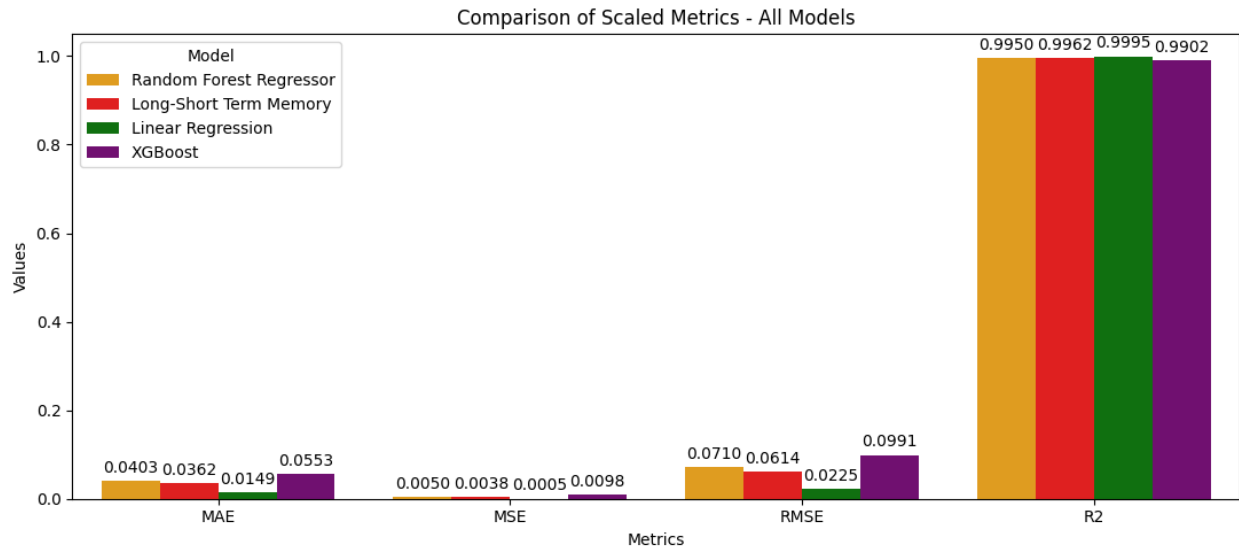
Slika 4-25 prikazuje usporedbu nekoliko prvih i zadnjih rezultata predviđanja LSTM modela.

	y_test	y_pred_lstm
Date		
2022-10-04	19623.580078	19097.134766
2022-10-05	20336.843750	19883.441406
2022-10-06	20160.716797	19759.556641
2022-10-07	19955.443359	19493.513672
2022-10-08	19546.849609	19024.501953
...	...	...
2024-08-24	64094.355469	62733.179688
2024-08-25	64178.992188	63208.242188
2024-08-26	64333.542969	63458.179688
2024-08-27	62880.660156	62875.203125
2024-08-28	59504.132812	60377.167969

Slika 4-25 Stvarne i predviđene vrijednosti - LSTM

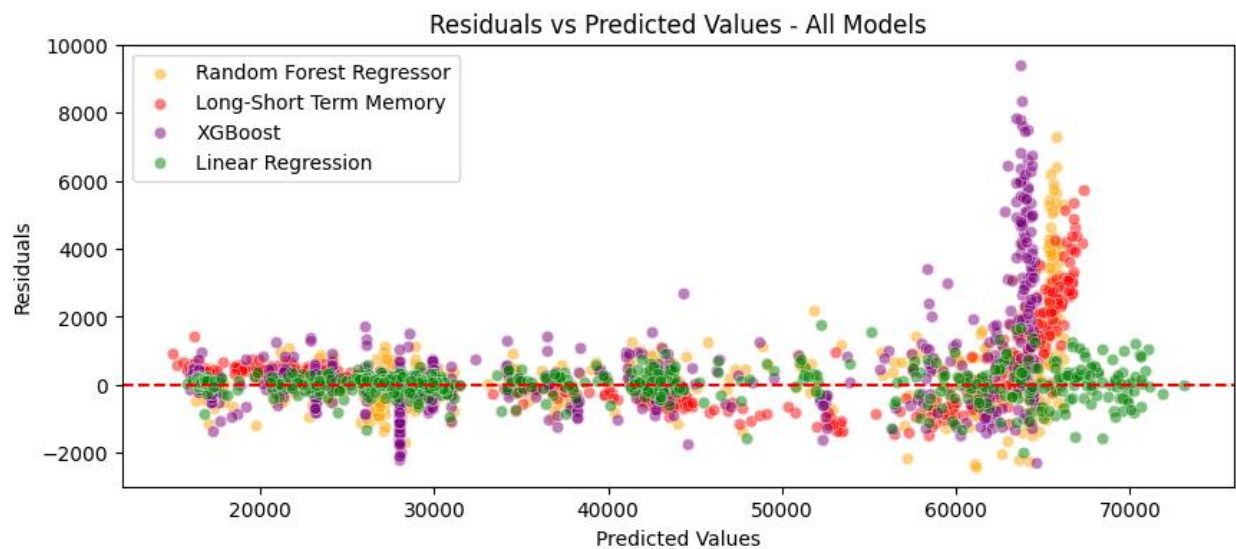
5 USPOREDBA REZULTATA

Slika 5-1 prikazuje usporedbu skaliranih rezultata evaluacijskih metrika za sve modele.



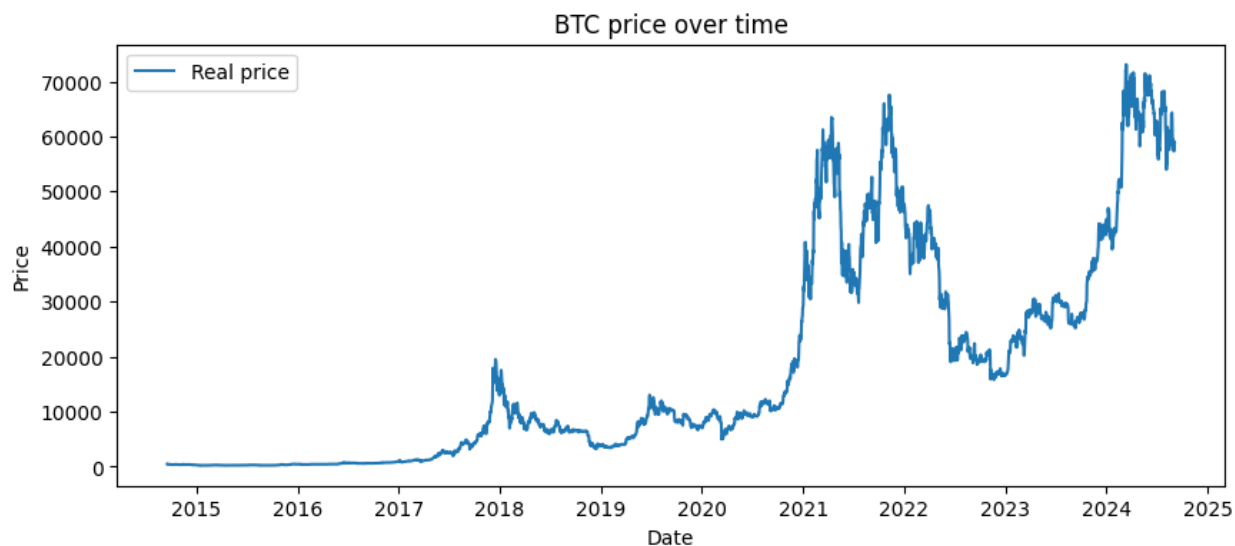
Slika 5-1 Skalirani rezultati evaluacijskih metrika (graf) – Svi modeli

Slika 5-2 prikazuje usporedbu reziduala svih modela.



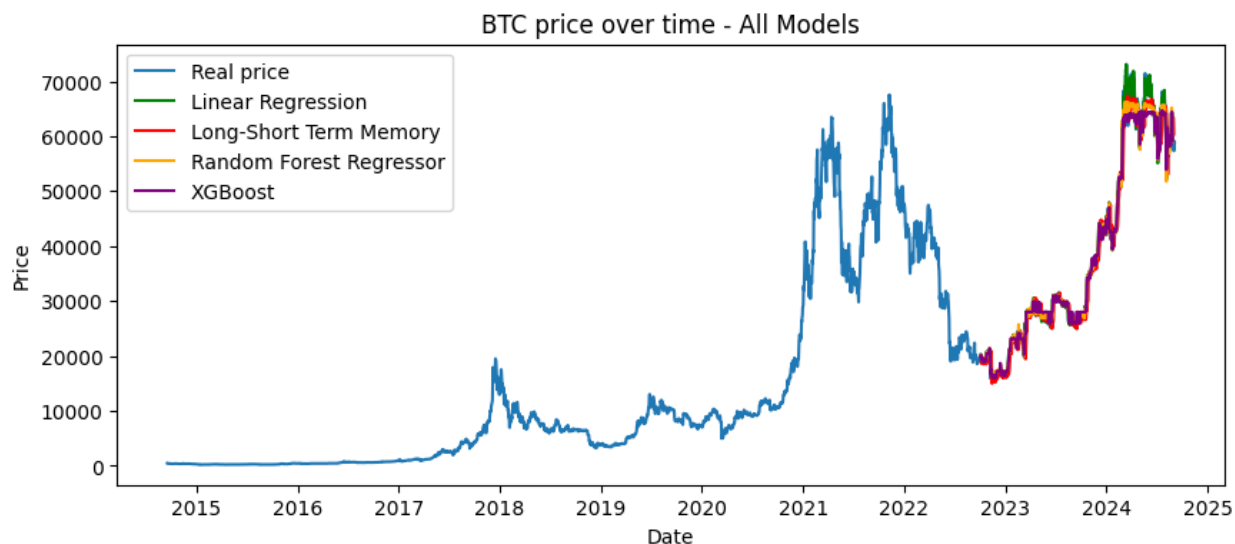
Slika 5-2 Reziduali – Svi modeli

Slika 5-3 prikazuje graf stvarnih „Close“ cijena tokom vremena.



Slika 5-3 Stvarne cijene – BTC

Slika 5-4 prikazuje graf na kojemu su uspoređene predviđene „Close“ cijene kroz vrijeme za sve modele. Kao referenca za ispravnost predviđanja može poslužiti slika 5-3.



Slika 5-4 Usporedba stvarnih i predviđenih cijena – Svi modeli

Iz priloženih grafova vidljivo je da je model linearne regresije postizao najbolje rezultate predviđanja. Naravno, postignuti rezultati nisu reprezentativni i treba ih uzeti s rezervom. Zbog ograničenih resursa nije ih bilo moguće usavršiti te je postizanje boljih rezultata i dalje moguće.

6 ZAKLJUČAK

Kao jedan od indikatora jačanja veze kripto valuta i strojnoga učenja je nastanak AI (eng. Artificial Intelligence) kripto valuta. AI kripto valute predstavljaju tokene integrirane u projekte koji se oslanjaju na tehnologije umjetne inteligencije. Razvoj „blockchain“ sustava baziranog na umjetnoj inteligenciji stvara robusniji i otporniji sustav za obavljanje transakcija. Takav sustav je omogućen AI algoritmima koji pronalaze najefikasnije rute za transakcije, prilagođavaju transakcijske naknade i tako štede resurse. Osim toga, stvaraju dodatni sloj zaštite stalnom analizom novih podataka i prepoznavanjem potencijalnih prevara. Uloga pripadnih AI tokena je da se koriste kao univerzalne digitalne valute za pokretanje takvih transakcija.

Iako nudi brojne mogućnosti, kripto svijet se i dalje suočava s problemom privlačenja pozornosti šire javnosti. Uzevši u obzir potencijal koji ova tehnologija nosi, broj korisnika je još uvijek relativno malen. Kao glavni razlozi ističu se opravdani skepticizam zbog visoke volatilnosti kripto valuta te česte internet prevare koje slijede ovu industriju.

S druge strane, strojno učenje postaje jedna od brže rastućih tehnologija. Svoju primjenu pronalazi u brojnim domenama pa se slobodno može reći da postaje neophodno kako za funkcioniranje strojeva tako i za svakodnevni život ljudi. Primjeni strojnog učenja u području kripto valuta se nudi prostor za daljnji napredak te inovacije, uključujući bolje algoritme za trgovanje, bolje procjene rizika i naprednije sigurnosne protokole.

U konačnici, simbioza kripto valuta i strojnog učenja može dovesti do transformacije globalnih financijskih tržišta stvarajući efikasniji i transparentniji sustav transakcija.

Literatura

- [1] <https://hrcak.srce.hr/file/382926>, s Interneta, 09.08.2024.
- [2] <https://smlbook.org/book/sml-book-draft-latest.pdf>, s Interneta, 09.08.2024.
- [3] <https://hrcak.srce.hr/file/459721>, s Interneta, 10.08.2024.
- [4] <https://pub.towardsai.net/bagging-vs-boosting-the-power-of-ensemble-methods-in-machine-learning-6404e33524e6>, s Interneta, 10.08.2024.
- [5] <https://hrcak.srce.hr/file/322233>, s Interneta, 10.08.2024.
- [6] <https://www.kaggle.com/code/ryanholbrook/a-single-neuron>, s Interneta, 10.08.2024.
- [7] <https://studymachinelearning.com/activation-functions-in-neural-network/>, s Interneta, 10.08.2024.
- [8] <https://srnghn.medium.com/deep-learning-overview-of-neurons-and-activation-functions-1d98286cf1e4>, s Interneta, 10.08.2024.
- [9] [http://alvarestech.com/temp/deep/Deep%20Learning%20by%20Ian%20Goodfellow,%20Yoshua%20Bengio,%20Aaron%20Courville%20\(z-lib.org\).pdf](http://alvarestech.com/temp/deep/Deep%20Learning%20by%20Ian%20Goodfellow,%20Yoshua%20Bengio,%20Aaron%20Courville%20(z-lib.org).pdf), s Interneta, 11.08.2024.
- [10] <https://www.math.mcgill.ca/yyang/resources/doc/randomforest.pdf>, s Interneta, 12.08.2024.
- [11] <https://www.saedsayad.com/docs/xgboost.pdf>, s Interneta, 13.08.2024.
- [12] <https://www.geeksforgeeks.org/deep-learning-introduction-to-long-short-term-memory/>, s Interneta, 16.08.2024.
- [13] https://www.researchgate.net/publication/373505006_Short-term_Traffic_Prediction_using_Deep_Learning_Long_Short-term_Memory_Taxonomy_Applications_Challenges_and_Future_Trends, s Interneta, 16.08.2024.
- [14] <https://www.geeksforgeeks.org/regression-metrics/>, s Interneta, 17.08.2024.
- [15] <https://research.google.com/colaboratory/faq.html>, s Interneta, 29.08.2024.
- [16] <https://mayerkrebs.com/yfinance-library-the-definitive-guide/>, s Interneta, 29.08.2024.
- [17] <https://flexiple.com/python/python-libraries>, s Interneta, 29.08.2024.
- [18] <https://finance.yahoo.com/quote/BTC-USD/history/>, s Interneta, 29.08.2024.

Sažetak

Zadatak ovog diplomskog rada bio je proučiti različite metode strojnog učenja u svrhu predviđanja cijena kripto valuta. Najprije je dan kratak uvod kako su nastale i čemu služe kripto valute te koja je njihova poveznica sa strojnim učenjem. U nastavku su objašnjeni osnovni koncepti odabranih metoda i evaluacijskih metrika, a potom je dan uvid u praktični dio rada. U praktičnom dijelu rada korištena su četiri različita modela predikcije, uključujući regresijske modele, modele vremenskih serija i modele temeljene na neuronskim mrežama. Implementacija modela je realizirana u Python programskom jeziku uz uporabu poznatih biblioteka kao što su Pandas, NumPy, scikit-learn, itd. Na kraju su prikazani i uspoređeni rezultati evaluacijskih metrika za sva četiri modela.

Ključne riječi:

Strojno učenje, bitcoin, neuron, algoritam , model

Summary

The task of this thesis was to study different machine learning methods for the purpose of predicting cryptocurrency prices. First, a brief introduction is given on how cryptocurrencies were created, what is their purpose and connection to machine learning. In the following sections the basic concepts of the selected methods and evaluation metrics are explained, followed by an insight into the practical part of the work. In the practical part of the work, four different predictive models were used, including regression models, time series models, and models based on neural networks. The implementation of the models was realized in the Python programming language with the use of well-known libraries such as Pandas, NumPy, scikit-learn, etc. Finally, the results of the evaluation metrics for all four models are presented and compared.

Title:

PREDICTIVE CRYPTOCURRENCY MODEL

Key words:

Machine learning, bitcoin, neuron, algorithm, model

Prilozi

Kazalo slika

Slika 2-1 Hijerarhija strojnog učenja [1]	3
Slika 2-2 Ansambl metode [4]	4
Slika 2-3 Neuron s jednim ulazom [6]	5
Slika 2-4 Neuron s više ulaza [6]	6
Slika 2-5 Sigmoidalna funkcija [7]	7
Slika 2-6 Tanh funkcija [7]	7
Slika 2-7 ReLU funkcija [7]	8
Slika 2-8 Neuronska mreža bez (lijevo) i s aktivacijskom funkcijom (desno) [8]	8
Slika 2-9 Duboka neuronska mreža [2]	10
Slika 4-1 Set podataka	24
Slika 4-2 Korelacija značajki	25
Slika 4-3 Odnos volumena i zaključne cijene	25
Slika 4-4 Prvi korak predobrade podataka	26
Slika 4-5 Testni skup podataka	27
Slika 4-6 Rezultati evaluacijskih metrika – LR	28
Slika 4-7 Skalirani rezultati evaluacijskih metrika (graf) – LR	28
Slika 4-8 Usporedba stvarnih i predviđenih cijena – LR	29
Slika 4-9 Reziduali – LR	29
Slika 4-10 Stvarne i predviđene vrijednosti – LR	30
Slika 4-11 Rezultati evaluacijskih metrika – RFR	31
Slika 4-12 Skalirani rezultati evaluacijskih metrika (graf) - RFR	31
Slika 4-13 Usporedba stvarnih i predviđenih cijena – RFR	32
Slika 4-14 Reziduali – RFR	32
Slika 4-15 Stvarne i predviđene vrijednosti – RFR	33
Slika 4-16 Rezultati evaluacijskih metrika – XGB	34
Slika 4-17 Skalirani rezultati evaluacijskih metrika (graf) – XGB	34
Slika 4-18 Usporedba stvarnih i predviđenih cijena – XGB	35
Slika 4-19 Reziduali – XGB	35

Slika 4-20 Stvarne i predviđene vrijednosti – XGB	36
Slika 4-21 Rezultati evaluacijskih metrika –LSTM.....	38
Slika 4-22 Skalirani rezultati evaluacijskih metrika (graf) – LSTM	39
Slika 4-23 Usporedba stvarnih i predviđenih cijena – LSTM.....	39
Slika 4-24 Reziduali – LSTM.....	40
Slika 4-25 Stvarne i predviđene vrijednosti - LSTM.....	40
Slika 5-1 Skalirani rezultati evaluacijskih metrika (graf) – Svi modeli.....	41
Slika 5-2 Reziduali – Svi modeli	41
Slika 5-3 Stvarne cijene – BTC	42
Slika 5-4 Usporedba stvarnih i predviđenih cijena – Svi modeli	42